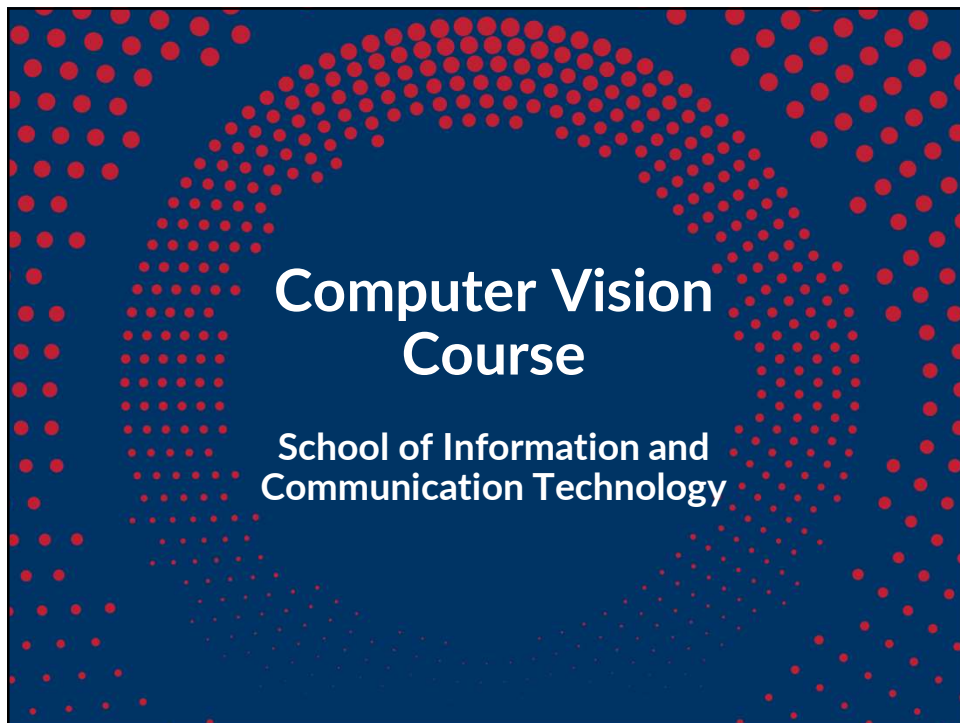




1



2




**ĐẠI HỌC
BÁCH KHOA HÀ NỘI**
 HANOI UNIVERSITY
 OF SCIENCE AND TECHNOLOGY

Computer Vision

Chapter 2. Image Processing (Part 1)

Computer Vision Group
 School of Information and Communications Technology

ONE LOVE. ONE FUTURE.

3



Contents

- Remind: digital image representation
- Point Operators
- Convolution and spatial filtering
- More neighborhood operators
- Image transforms

 hust.edu.vn
 fb.com/dhbkhn

CV Group – School of Information and Communication and Technology

4

Digital images ?

- What we can see on the picture?

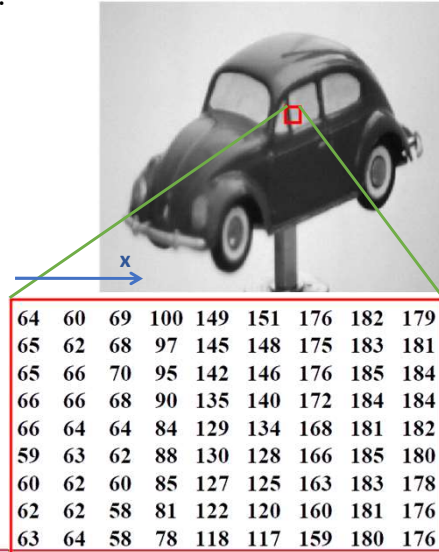
- A car?

- What does the machine see?

- Image is a matrix of pixels
- Image $N \times M$: $N \times M$ matrix
- 1 pixel (gray levels):
 - A intensity value: 0-255
 - Black: 0
 - White: 255

- For an image I

- Index (0,0): Top left corner
- $I(x,y)$: intensity of pixel at the position (x,y)



Digital images ?

- Principal type of images

- Binary image:

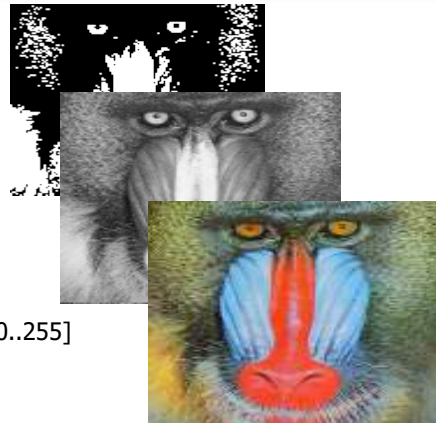
- $I(x,y) \in \{0, 1\}$
- 1 pixel: 1 bit

- Gray image:

- $I(x,y) \in [0..255]$
- 1 pixel: 8 bits (1 byte)

- Color image

- $I_R(x,y), I_G(x,y), I_B(x,y) \in [0..255]$
- 1 pixel: 24 bits (3 bytes)



- Other : multi-spectral images, depth images, etc.

Pixel transformation

f

g

Isolated: $g(x_0, y_0) = T[f(x_0, y_0)]$

Local: $g(x_0, y_0) = T[f(V)]$
 V : neighbors of (x_0, y_0)

Global: $g(x_0, y_0) = T[f(x, y)]$
 example: FFT

Source : Caroline Rougier. Traitement d'images (IFT2730). Univ. de Montréal.

7

7

HUST

hust.edu.vn
 fb.com/dhbkhn

Contents

- Remind: digital image representation
- Point Operators
- Convolution and spatial filtering
- More neighborhood operators
- Image transforms

hust.edu.vn
 fb.com/dhbkhn

CV Group – School of Information and Communication and Technology

8

Point processing

- The output at one point depends only on that point but not other neighborhoods

$$s = T(r)$$

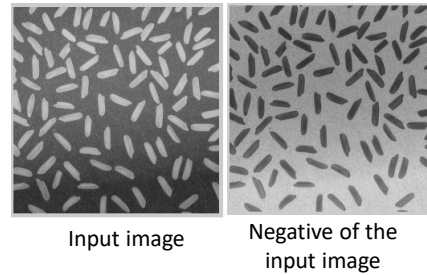
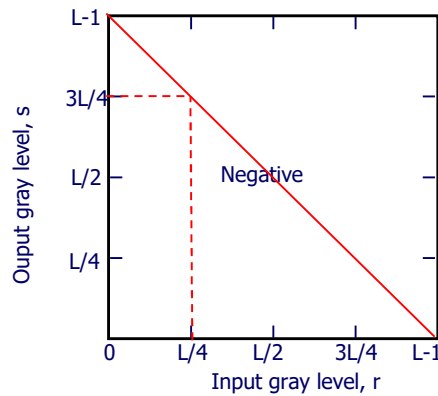
- $r = f(x, y)$
- $s = g(x, y)$
- T: transformation function

Some Basic Gray Level Transformations

- Image Negatives
- Log Transformations
- Power-Law Transformations
- Piecewise-Linear Transformation Functions
- Histogram Processing

Image Negatives

- The negative of an image with gray levels in the range $[0, L-1]$ is obtained by $s = L - 1 - r$



11

Log Transformations

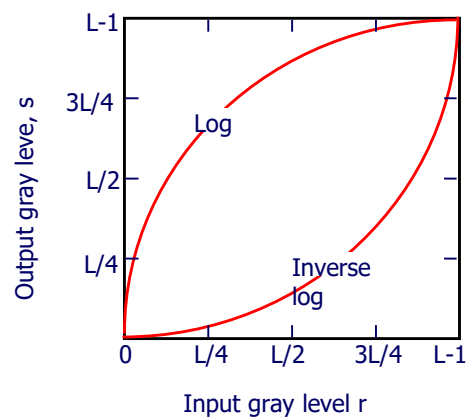
- The general form of the log transformation:

$$s = c \times \log(1 + r)$$

■ c : constant

■ $r \geq 0$

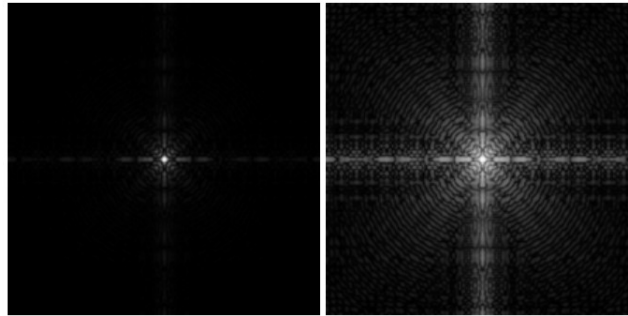
m rng giá tr ca các im nh ti
trong hình nh trong khi nén
các giá tr cp cao hn



12

Log Transformations

- Expand the values of dark pixels in an image while compressing the higher-level values

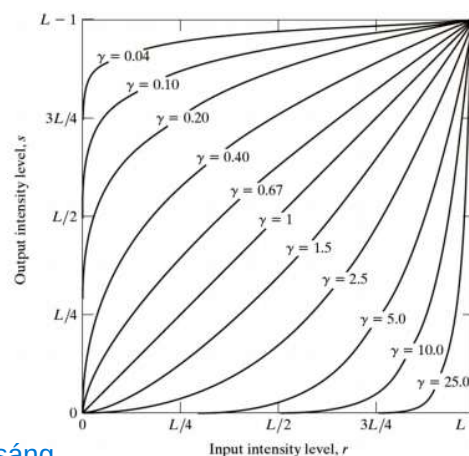


13

Power-Law (Gamma) Transformations

- The general form of power-law transformation is:
 - $\gamma > 1$: compress values in dark area, while expanding values in light area
 - $\gamma < 1$: expand values in dark area, while compressing values in light area
 - r : normalized values to $[0, 1]$
 - c : scaling constant c corresponding to the bit size used

gamma>1: gim chói, gim cháy sáng
gamma<1: nén tỉ, tng sáng



14

Power-Law (Gamma) Transformations

- $\gamma_1 = 1$; $\gamma_2 = 3$; $\gamma_3 = 4$; $\gamma_4 = 5$;



Increasing the dynamic ranges of high intensities

15

Power-Law Transformations

- Given input image I , intensities are mapped into $[0, 255]$. Use $s = r^{0.3}$ to find output image

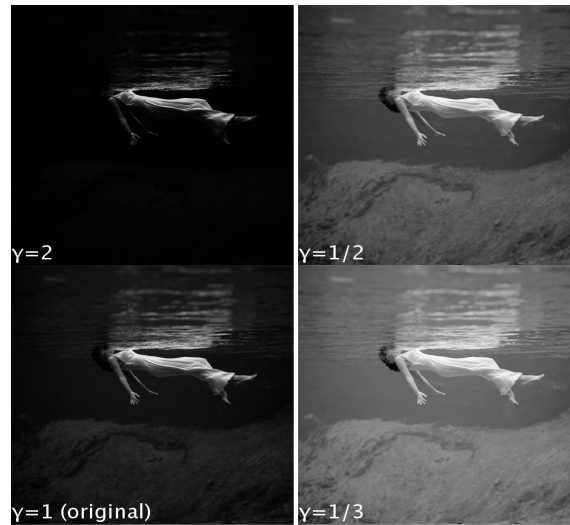
24	25	45	18	90	45	54
16	2	25	214	97	54	54
18	154	14	201	98	65	54
19	254	13	201	50	31	24
200	210	254	231	47	201	8
21	218	217	120	102	156	58
0	236	208	10	12	95	4

→

3	3	3	2	4	3	3
2	1	3	5	4	3	3
2	5	2	5	4	3	3
2	5	2	5	3	3	3
5	5	5	5	3	5	2
2	5	5	4	4	5	3
0	5	5	2	2	4	2

16

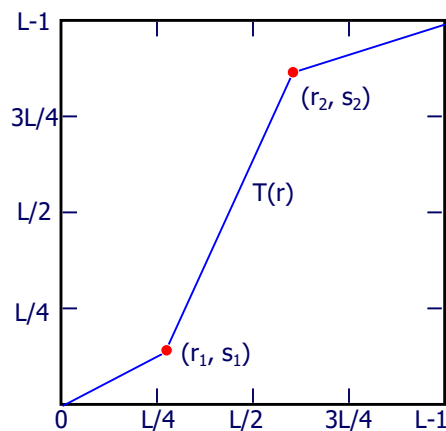
Gama correction



17

Piecewise-Linear Transformation

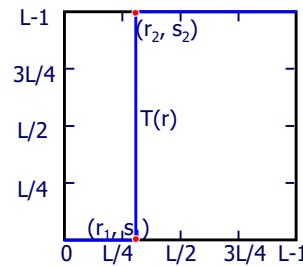
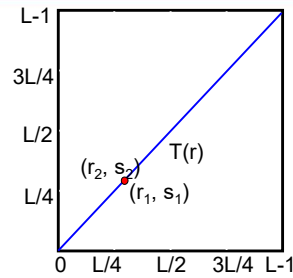
- Contrast stretching:
 - the idea behind contrast stretching is to **increase the dynamic range** of the gray levels in the image being processed
- Output:
 - Output values of input ones from $(0, 0)$ to (r_1, s_1) decrease
 - Output values of input ones from (r_1, s_1) đến (r_2, s_2) increase.



18

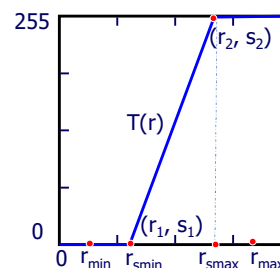
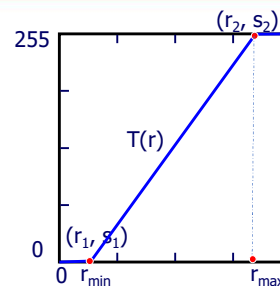
Piecewise-Linear Transformation

- If $r_1 = s_1, r_2 = s_2$,
 - the function becomes Homogenous transformation
- If $s_1 = 0, s_2 = L-1$,
 - the function become thresholding function
 - Output is a binary image



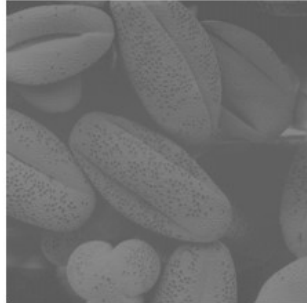
Piecewise-Linear Transformation

- If $(r_1, s_1) = (r_{\min}, 0)$;
 $(r_2, s_2) = (r_{\max}, 255)$
 - Linear stretching
- If $(r_1, s_1) = (r_{\min}, 0), r_{\min} > r_{\min}$
 $(r_2, s_2) = (r_{\max}, 255), r_{\max} < r_{\max}$
 - Linear stretching with saturation

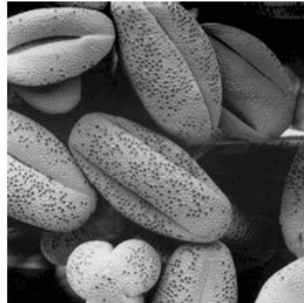


ct ngng

Piecewise-Linear Transformation



Original image, low contrast



Contrast sketching
 $(r_1, s_1) = (r_{\min}, 0)$
 $(r_2, s_2) = (r_{\max}, L-1)$

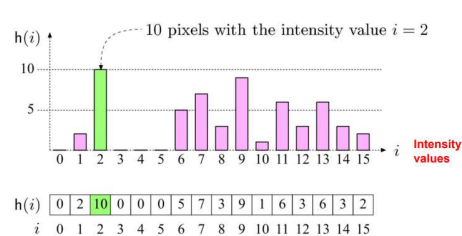


Thresholding
 $(r_1, s_1) = (m, 0)$
 $(r_2, s_2) = (m, L-1)$
 m : mean of intensities

21

Image histogram

- What is histogram?
 - Histogram of an image with gray level range $[0, L-1]$ is a discrete function: $h(r_k) = n_k$
 - r_k is k^{th} gray level
 - n_k : number of pixels that have level r_k
 - Histogram is a graphical representation of the repartition of colours among the pixels of a digital image



22

Image histogram

• Normalized histogram?

- The histogram should be normalized by dividing all elements to total number of pixels in the image (n): $h(r_k) = \frac{n_k}{n}$

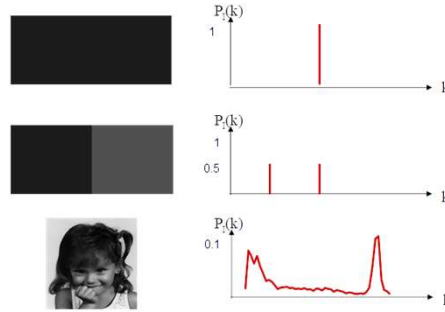


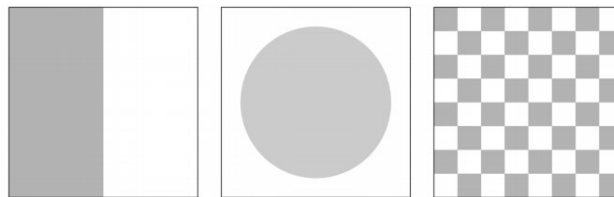
Image dynamic range = [min intensity, max intensity]

23

Image histogram

• Properties?

- Only statistic information
- No indication about the location of pixel (no spatial information)
- Different images can have the same histogram



24

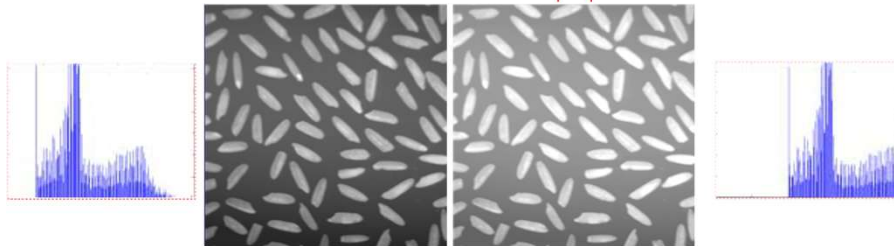
Image Brightness

- Brightness of a grayscale image is the average intensity of all pixels in an image
 - refers to the overall lightness or darkness of the image

$$B(I) = \frac{1}{wh} \sum_{v=1}^h \sum_{u=1}^w I(u, v)$$

Divide by total number of pixels

Sum up all pixel intensities



Contrast

- The contrast of a grayscale image indicate how easily object in the image can be distinguished
- Many different equations for contrast exist
 - Standard deviation of intensity values of pixels in the image

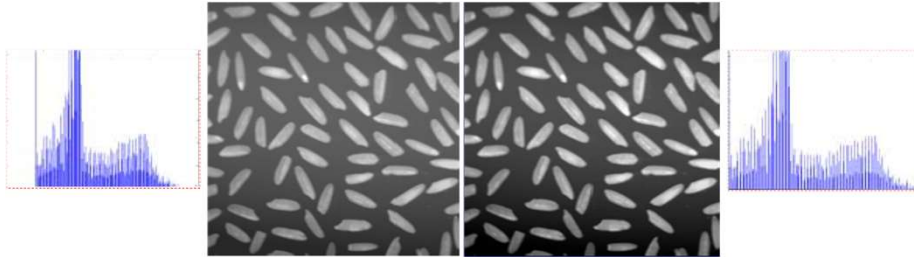
$$C = \sqrt{\frac{1}{wh} \sum_{u=1}^h \sum_{v=1}^w (I(u, v) - \text{mean})^2}$$

- Difference between intensity value maximum et minimum

$$C = \frac{\max(I(u, v)) - \min(I(u, v))}{\max(I(u, v)) + \min(I(u, v))}$$

Contrast

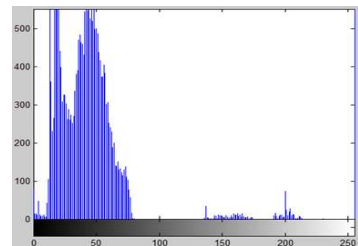
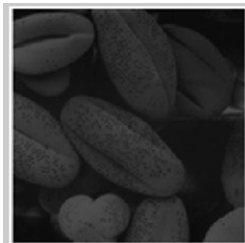
- Contrast vs histogram



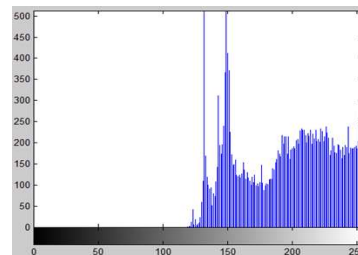
27

Histogram and Brightness

Dark image



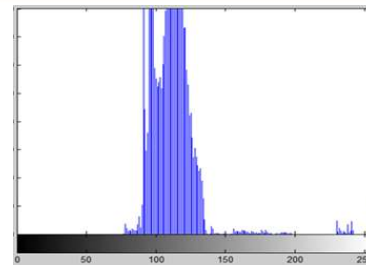
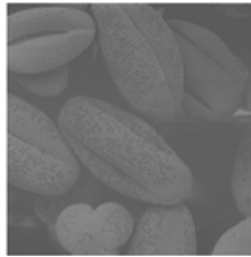
Light image



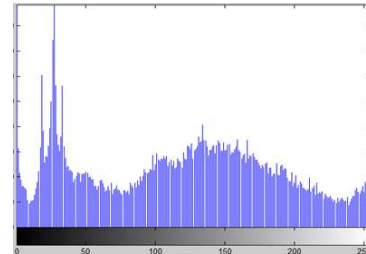
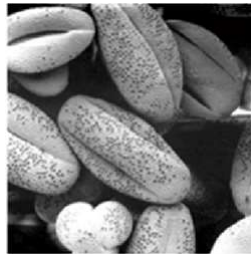
28

Histogram and Contrast

Low- contrast
image



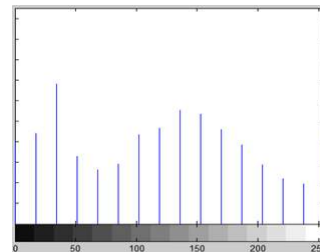
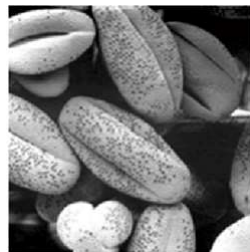
High- contrast
image



29

Histogram

- If we can somehow equalize the histogram of an image → we can make it higher contrast.



30

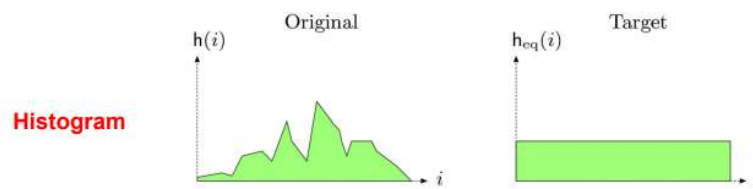
Histogram equalization

- **Input:**

- Image I which have L gray levels $r_k \in [0, L-1]$, $k=0, 1, \dots, L-1$. Total pixels is n .
- Assume that the input image is low contrast

- **Output:**

- Image J with gray level $s_k \in [0, L-1]$, $k=0, 1, \dots, L-1$
- Histogram of modified image: **uniform distribution**
- Each gray level in the image occurs with the same frequency



31

Histogram equalization

- **Step-by-step**

- s1. count the number of pixels which have gray level k in image, let's say n_k ($k=0, 1, \dots, L-1$)

- s2. compute $p(k)$ like follows
(normalized histogram):

$$p(k) = \frac{n_k}{n}$$

$$k = 0, 1, 2, \dots, L-1.$$

32

Histogram equalization

- s3. Compute $T(k)$ (cumulative histogram)

$$T(k) = \sum_{j=0}^k p(j) = \frac{1}{n} \sum_{j=0}^k n_j \quad k = 0, 1, 2, \dots, L-1.$$

- s4. Gray level s_k of output image, J , responsible to gray level k of input image is calculated by:

$$s_k = \text{round}[(L-1)T(k)] \quad k = 0, 1, 2, \dots, L-1.$$

- s5. Build the output image by replacing the gray level k by s_k .

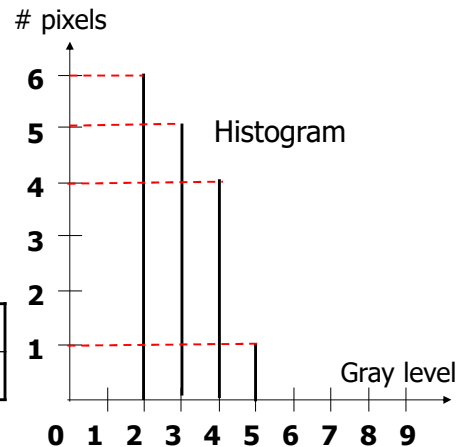
33

Histogram equalization

Ex. 1. Input image I has 10 levels.
Equalize the histogram of I

2	3	3	2
4	2	4	3
3	2	3	5
2	4	2	4

k	0	1	2	3	4	5	6	7	8	9
n_k	0	0	6	5	4	1	0	0	0	0



34

Histogram equalization

k	0	1	2	3	4	5	6	7	8	9
n_k	0	0	6	5	4	1	0	0	0	0
$\sum_{j=0}^k n_j$	0	0	6	11	15	16	16	16	16	16
$T(k)$	0	0	6/16	11/16	15/16	16/16	16/16	16/16	16/16	16/16
$s_k = 9 * T(k)$	0	0	3.3 ≈3	6.1 ≈6	8.4 ≈8	9	9	9	9	9

HUST
HÀNG UNIVERSITY OF SCIENCE AND TECHNOLOGY

35

35

Histogram equalization

k	s_k
0	0
1	0
2	3
3	6
4	8
5	9
6	9
7	9
8	9
9	9

2	3	3	2
4	2	4	3
3	2	3	5
2	4	2	4

↓

3	6	6	3
8	3	8	6
6	3	6	9
3	8	3	8

pixels

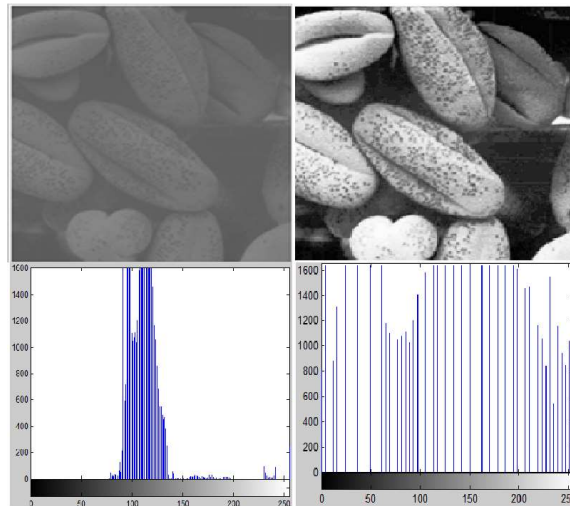
Histogram after equalization

HUST
HÀNG UNIVERSITY OF SCIENCE AND TECHNOLOGY

36

36

Histogram equalization



37

Histogram equalization

If we take the **same image** with **different contrasts**, histogram equalization will give the **same results** for all images

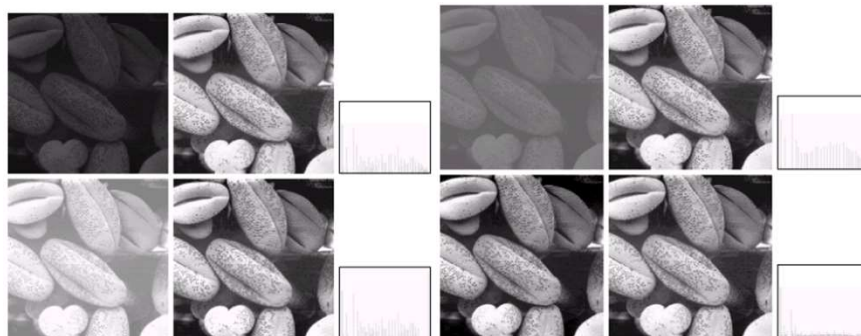
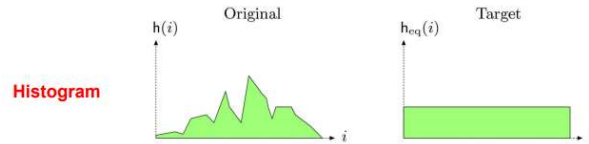


FIGURE 3.17 (a) Images from Fig. 3.15. (b) Results of histogram equalization. (c) Corresponding histograms.

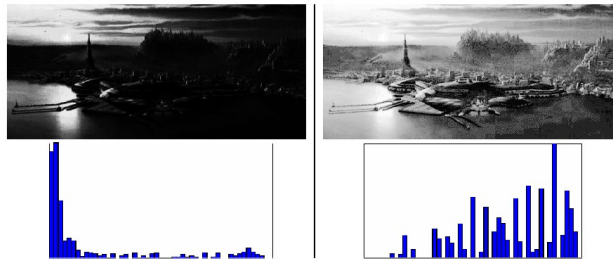
38

Histogram equalization

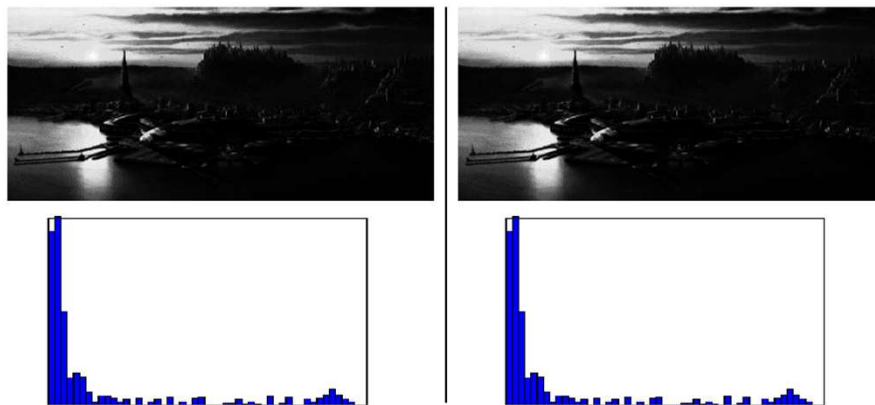
- Change histogram of modified image into uniform distribution



- No parameters. OpenCV: `cv2.equalizeHist(img)`



Linear stretching

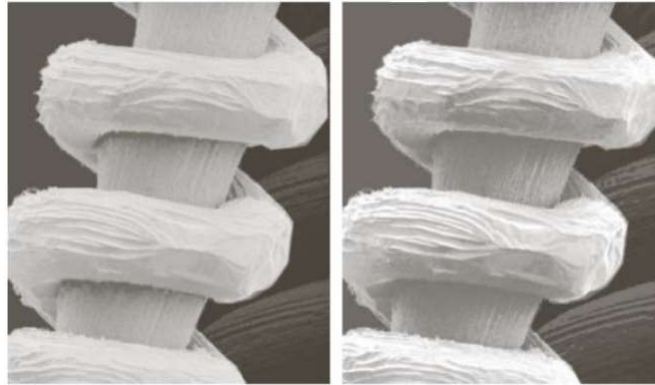


No differences?

Histogram statistic

Original image

After histogram equalization



41

Contents

- Remind: digital image representation
- Point Operators
- Convolution and spatial filtering
- More neighborhood operators
- Image transforms

42

Image filtering

- Image filtering:
 - **Image filters in spatial domain (called spatial filtering)**
 - Filter is a mathematical operation of a grid of numbers
 - Smoothing, sharpening, measuring texture, ...
 - Image filters in the frequency domain (**later**)
 - Filtering is a way to modify the frequencies of images
 - Denoising, sampling, image compression,...
- Really important!
 - Enhance images: Denoise, smooth, increase contrast, etc.
 - Extract information from images:
 - Texture, edges, distinctive points, etc.
 - Detect patterns
 - Template matching

43

Convolution and spatial filtering

- Spatial filtering
 - affects directly to pixels in image
 - Local transformation in the spatial domain can be represented as:

$$g(x,y) = T(f(x,y))$$
 - T may affect the point (x,y) and its neighborhood (K) and output a new value
 (K: Filter/Mask/Kernel/Window/Template Processing)
 - **Same function** applied at each position
 - Output and input image are typically **the same size**
- Convolution: Linear filtering, function is a weighted sum/difference of pixel values

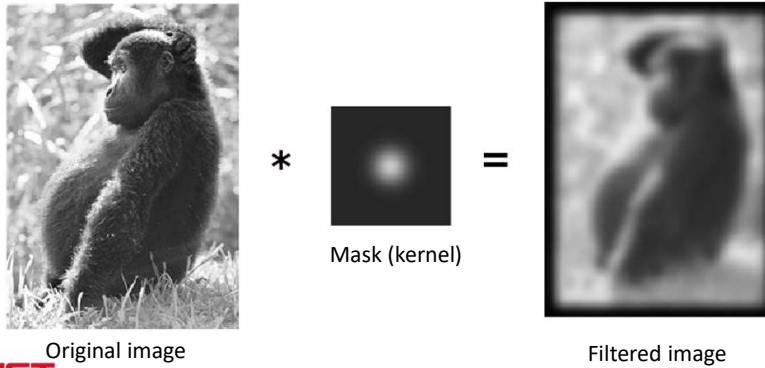
$$I' = I * K$$

44

Spatial Convolution

- Convolution: Linear filtering, function is a weighted sum/difference of pixel values

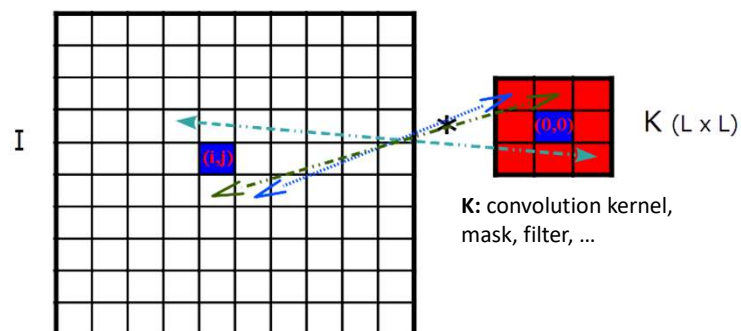
$$f(n, m) = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} f[k, l] \times h[n - k, m - l]$$



45

Spatial convolution

- New value of a pixel(i,j) is a weighted sum of its neighbors

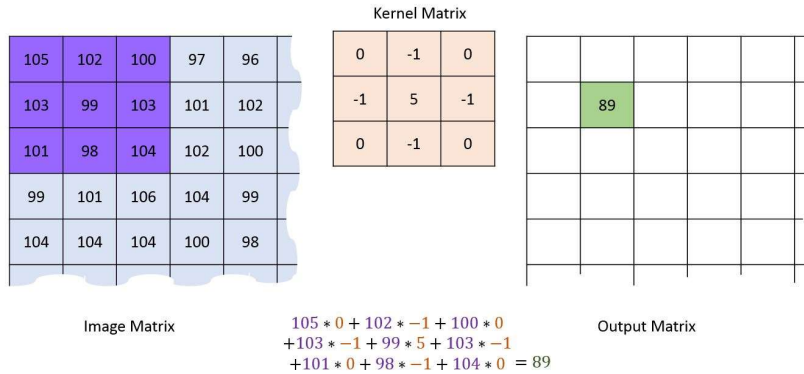


$$I'(i, j) = \sum_{u=-\frac{(L-1)}{2}}^{\frac{(L-1)}{2}} \sum_{v=-\frac{(L-1)}{2}}^{\frac{(L-1)}{2}} I(i-u, j-v) K(u, v)$$

46

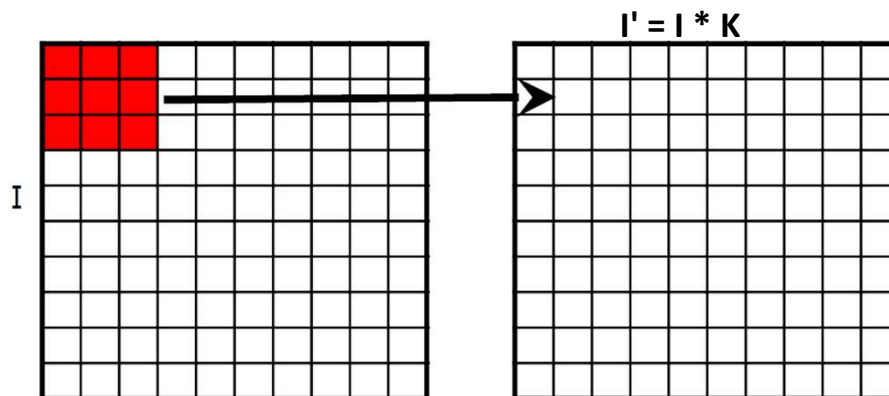
Spatial convolution

- New value of a pixel(i,j) is a weighted sum of its neighbors

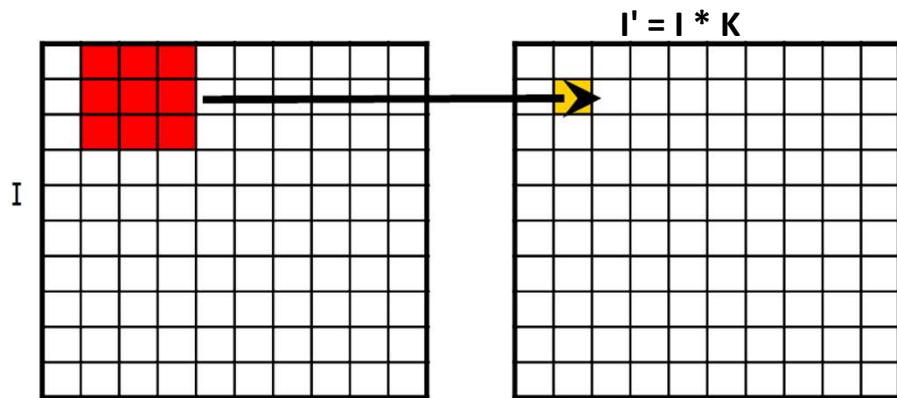


Source: <http://machinelearninguru.com>

Spatial convolution

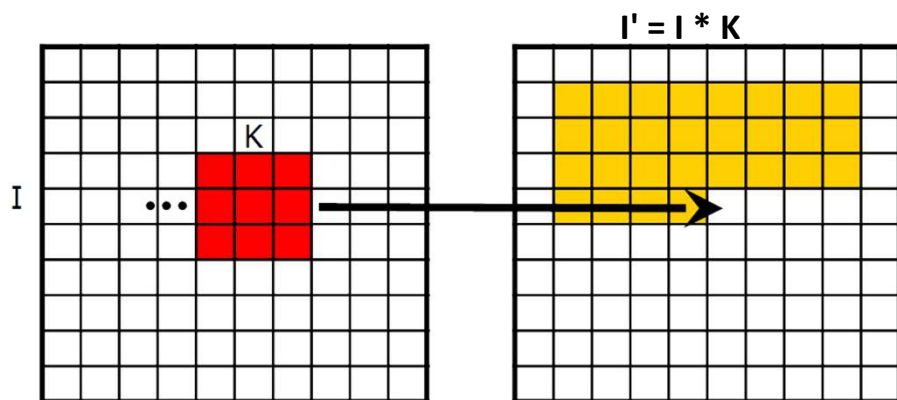


Spatial convolution



49

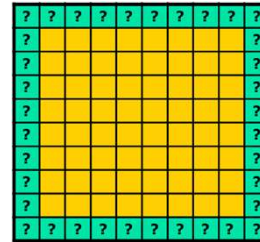
Spatial convolution



50

Spatial convolution

- Border problem?
 - Ignore, set to 0
 - Zero padding to the input matrix
 - reflect across edge:
 - $f(-x,y) = f(x,y)$
 - $f(-x,-y) = f(x,y)$
 - ...



0	0	0	0	0	0
0	105	102	100	97	96
0	103	99	103	101	102
0	101	98	104	102	100
0	99	101	106	104	99
0	104	104	104	100	98

105	105	102	100	97	96
105	105	102	100	97	96
103	103	99	103	101	102
101	101	98	104	102	100
99	99	101	106	104	99
104	104	104	104	100	98



51

Spatial convolution

0	0	0	0	0	0
0	105	102	100	97	96
0	103	99	103	101	102
0	101	98	104	102	100
0	99	101	106	104	99
0	104	104	104	100	98

Kernel Matrix		
0	-1	0
-1	5	-1
0	-1	0

320				
210	89	111		

Image Matrix

$$\begin{aligned}
 &0 \times 0 + 0 \times -1 + 0 \times 0 \\
 &+ 0 \times -1 + 105 \times 5 + 102 \times -1 \\
 &+ 0 \times 0 + 103 \times -1 + 99 \times 0 = 320
 \end{aligned}$$

Output Matrix

105	105	102	100	97	96
105	105	102	100	97	96
103	103	99	103	101	102
101	101	98	104	102	100
99	99	101	106	104	99
104	104	104	104	100	98

Kernel Matrix

0	-1	0
-1	5	-1
0	-1	0

110				
	89	111		

Image Matrix

Output Matrix

52

Some kernels

- 2D spatial convolution
 - is mostly used in image processing for feature extraction
 - And is also the core block of Convolutional Neural Networks (CNNs)
- Each kernel has its own effect and is useful for a specific task such as
 - blurring (noise removing),
 - sharpening,
 - edge detection,
 -

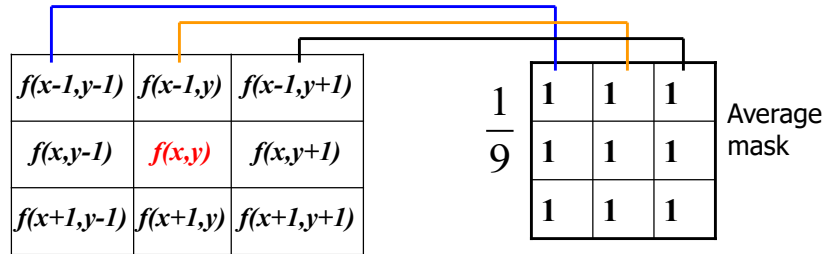
53

Smooth filtering

- Application
 - Noise remove
 - Image blurring
- Average filtering is the simplest of smooth filtering

54

Average (mean) filtering



$$g(x, y) = \frac{1}{9} [f(x-1, y-1) + f(x-1, y) + f(x-1, y+1) + f(x, y-1) + f(x, y) + f(x, y+1) + f(x+1, y-1) + f(x+1, y) + f(x+1, y+1)]$$

Average (mean) filtering

- Mask:
 - All elements of the mask are equal
- Results:
 - Replacing each pixel with an average of its neighborhood
 - Achieve smoothing effect

$$\frac{1}{9} \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



Original image

Filtered image
with box size 5x5Filtered image
with box size 11x11

Smooth filtering

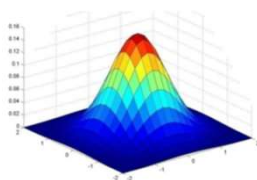
- General formulation

$$g(x, y) = \frac{\sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)}{\sum_{s=-a}^a \sum_{t=-b}^b w(s, t)}$$

- To avoid modifying the overall luminance of the image, the sum of the coefficients must be equal to 1

57

Gaussian filtering



Gaussian function in 3D



Gaussian image

0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

Gaussian filter with size 5 x 5, sigma = 1

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

Rule for Gaussian filter:
set filter width to about 6σ or 8σ [+1]

58

Gaussian filtering



Original image



Filtered image
with box size 5x5

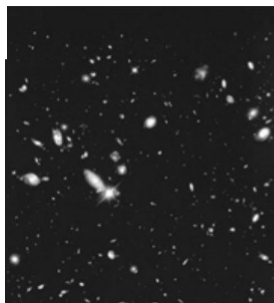


Filtered image
with box size 11x11

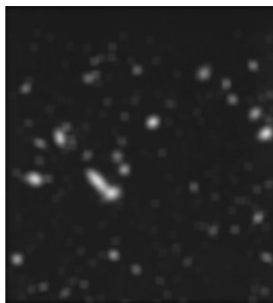
Smooth filtering

- Blurring usage: delete unwanted (small) subjects

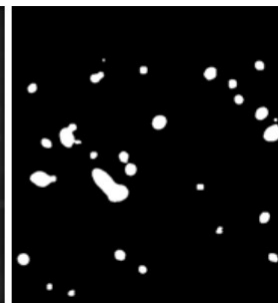
Original image



Average filtering: 15x15



Thresholding of blurring image



Sharpening filter

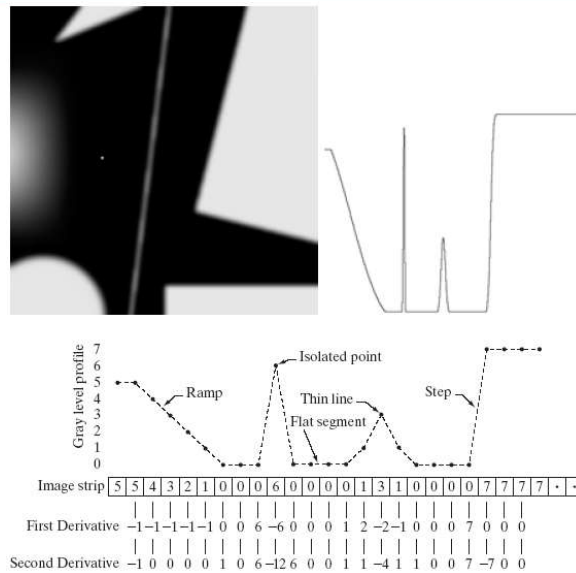
- Highlights Intensity Transitions
- Base on first and second order derivatives

$$\frac{\partial f}{\partial x} \approx \begin{cases} f(x+1, y) - f(x, y) \\ f(x, y) - f(x-1, y) \\ 0.5(f(x+1, y) - f(x-1, y)) \end{cases}$$

$$\frac{\partial^2 f}{\partial x^2} \approx f(x+1, y) - 2f(x, y) + f(x-1, y)$$

61

Example of first and second derivatives



62

Derivative

- 1st and 2nd Order Derivative Comparison:
 - First Derivative:
 - Thicker Edge;
 - Strong Response for step changes;
 - Second Derivative:
 - Strong response for fine details and isolated points;
 - Double response at step changes

63

First derivatives

- Filters used to compute the first derivatives of image

- Robert

1	0
0	-1

Robert kernels

- Prewitt

- less sensitive to noise
- Smoothing with mean filter,

then compute 1st derivative

- Sobel:

- less sensitive to noise
- Smoothing with gaussian,

then computing 1st derivative

-1	-1	-1
0	0	0
1	1	1

Prewitt kernels

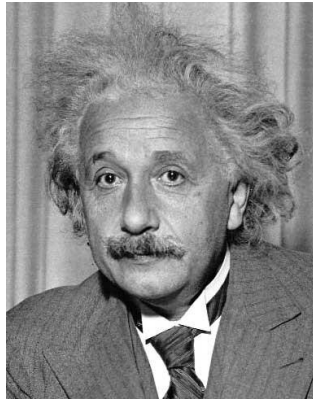
-1	-2	-1
0	0	0
1	2	1

Sobel kernels

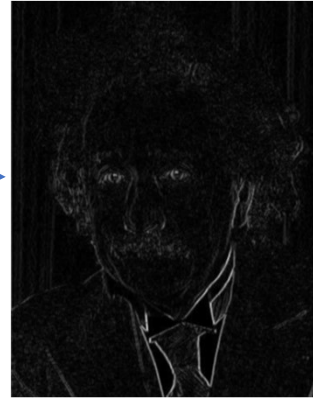
64

First derivatives

- Sobel



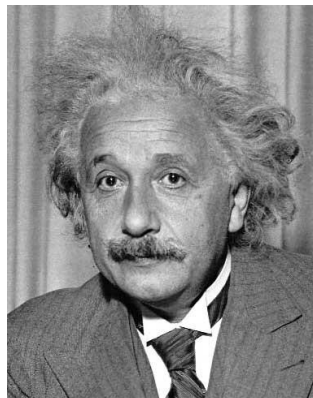
$$* \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$



Vertical Edge
(absolute value)

First derivatives

- Sobel



$$* \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$



Horizontal Edge
(absolute value)

2nd derivatives - Laplacian filtering

- The Laplacian operator is defined as:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$\frac{\partial^2 f}{\partial x^2} = f(x+1, y) + f(x-1, y) - 2f(x, y)$$

$$\frac{\partial^2 f}{\partial y^2} = f(x, y+1) + f(x, y-1) - 2f(x, y)$$

2nd derivatives - Laplacian filtering

- Can be computed as

$$\nabla^2 f = [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1)] - 4f(x, y)$$

0	1	0
1	-4	1
0	1	0

- Or

$$\nabla^2 f = 4f(x, y) - [f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1)]$$

0	-1	0
-1	4	-1
0	-1	0

- 90° isotropic filter

Sharpening filter using laplacian

$$g(x, y) = \begin{cases} f(x, y) - \nabla^2 f(x, y) \\ f(x, y) + \nabla^2 f(x, y) \end{cases}$$

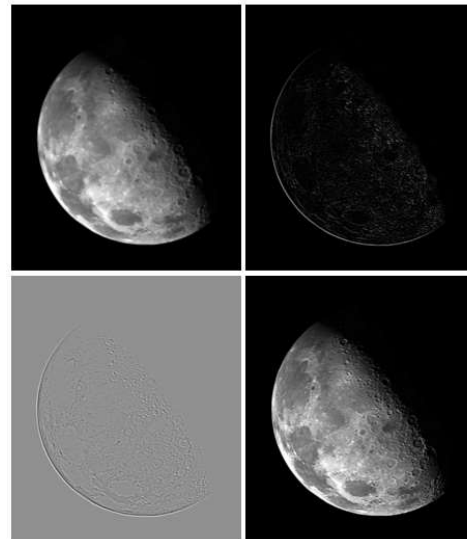
$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

69

Sharpening filter

- a) Original image
- b) Laplacian filtering
- c) Laplacian with scaling
- d) Adding original with Laplacian image

FIGURE 3.40
(a) Image of the North Pole of the moon.
(b) Laplacian-filtered image.
(c) Laplacian image scaled for display purposes.
(d) Image enhanced by using Eq. (3.7-5).
(Original image courtesy of NASA.)



70



Contents

- Remind: digital image representation
- Point Operators
- Convolution and spatial filtering
- More neighborhood operators
- Image transforms

hust.edu.vn
fb.com/dhbkhn

CV Group – School of Information and Communication and Technology

71

Median filter

- No new pixel values introduced
- Removes spikes:
 - good for impulse, salt & pepper noise
- Non-linear filter

10	15	20
23	90	27
33	31	30

Median value →

10	15	20	23	27	30	31	33	90
----	----	----	----	----	----	----	----	----

↓ Sort

10	15	20
23	27	27
33	31	30

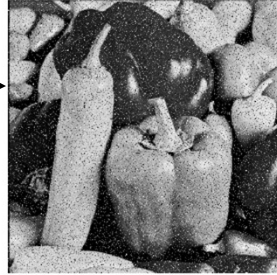
↓ Replace

10	15	20
23	27	27
33	31	30

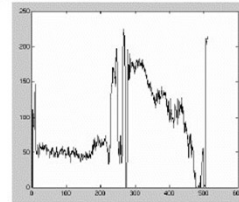
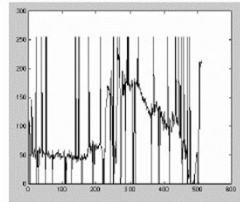
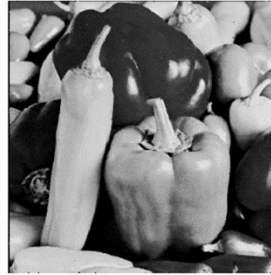
72

Median filter

Salt and
pepper
noise



Median
filtered



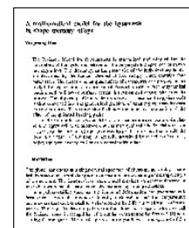
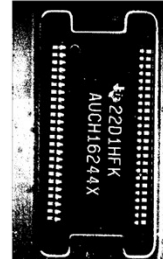
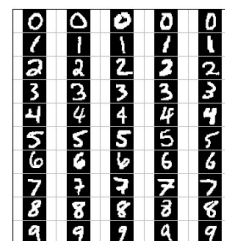
Plots of a row of the image

HUST
HÀNG HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Source: M. Hebert

73

Binary images



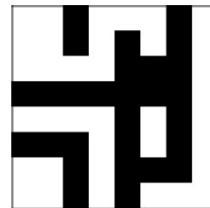
HUST
HÀNG HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

74

Binary images

- Two pixel values: foreground (object, 1) and background (0)
- Be used
 - To mark region(s) of interest
 - As results of thresholding method

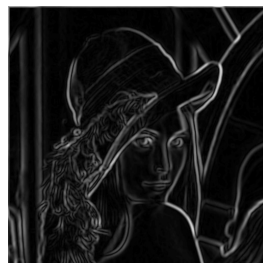
1	1	0	1	1	1	0	1
1	1	0	1	0	1	0	1
1	1	1	1	0	0	0	1
0	0	0	0	0	0	0	1
1	1	1	1	0	1	0	1
0	0	0	1	0	1	0	1
1	1	0	1	0	0	0	1
1	1	0	1	0	1	1	1



Thresholding

- Given a grayscale image or an intermediate matrix → threshold to create a binary output.

Example: edge detection



Gradient magnitude



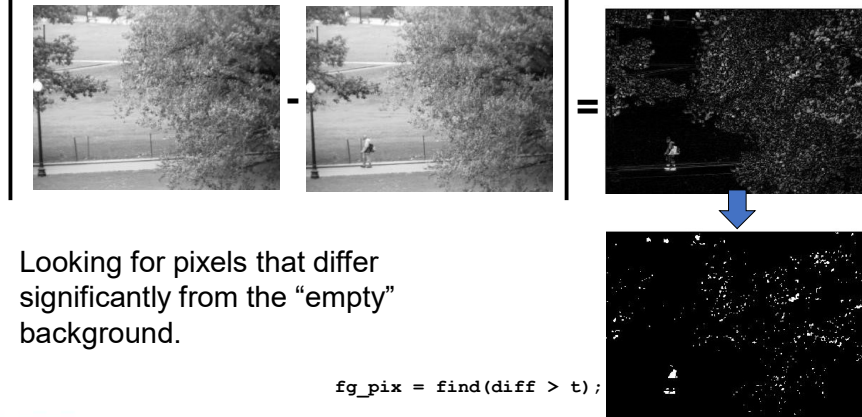
`fg_pix = find(gradient_mag > t);`

Looking for pixels where gradient is strong.

Thresholding

- Given a grayscale image or an intermediate matrix → threshold to create a binary output.

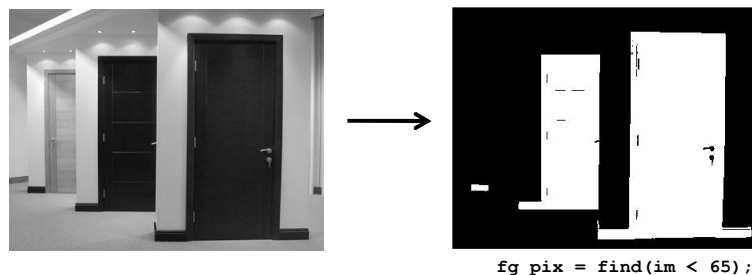
Example: background subtraction



Thresholding

- Given a grayscale image or an intermediate matrix → threshold to create a binary output.

Example: intensity-based detection

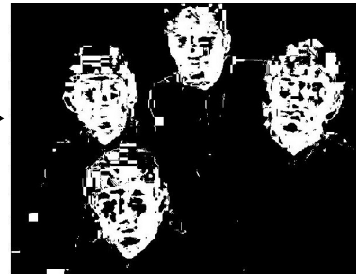


Looking for dark pixels

Thresholding

- Given a grayscale image or an intermediate matrix → threshold to create a binary output.

Example: color-based detection

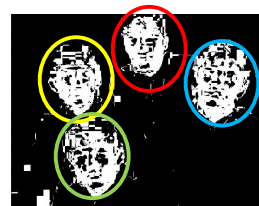


```
fg_pix = find(hue > t1 & hue < t2);
```

Looking for pixels within a certain hue range.

Issues

- What to do with “noisy” binary outputs?
 - Holes
 - Extra small fragments
- How to demarcate multiple regions of interest?
 - Count objects
 - Compute further features per object

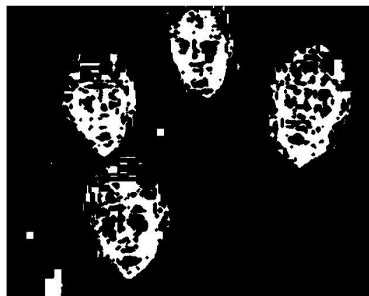


Morphological operators

- Change the shape of the foreground regions via intersection/union operations between a scanning structuring element and binary image.
- Useful to clean up result from thresholding
- Main components
 - Structuring element
 - Operators:
 - Basic operators: Dilation, Erosion
 - Others: Opening, Closing, ...

Dilation

- Expands connected components
- Grow features
- Fill holes



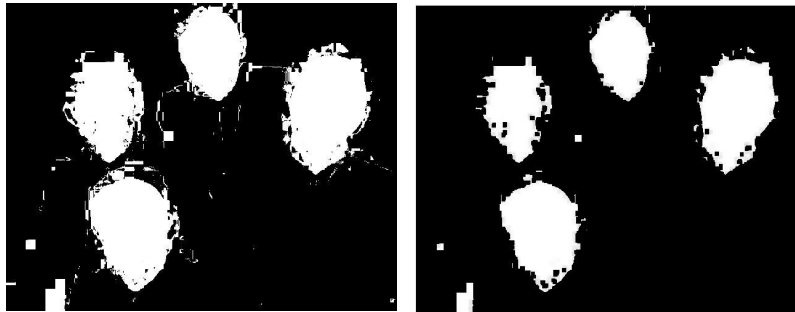
Before dilation



After dilation

Erosion

- Erode connected components
- Shrink features
- Remove bridges, branches, noise



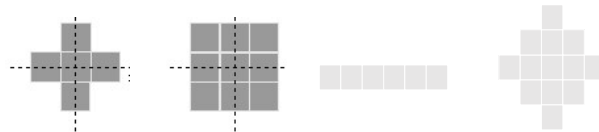
Before erosion

After erosion

83

Structuring elements

- **Masks** of varying shapes and sizes used to perform morphology, for example:

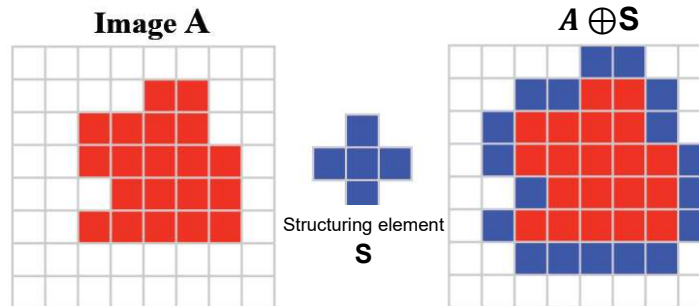


- Scan mask (structuring element) over the **object (foreground) borders (inside and outside)** and transform the binary image

84

Dilation

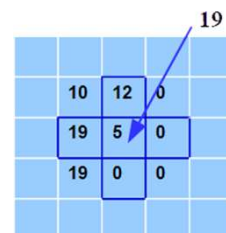
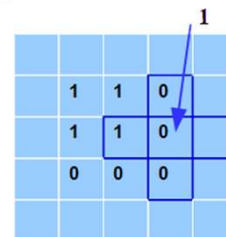
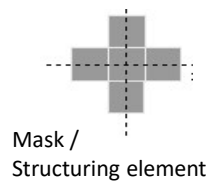
- Moving S on each pixel of A
 - check if the intersection (pixels belonging to object) is not empty
 - If yes, the center of B belongs to the result image
- If a pixel of S is onto object pixels (A), then the central pixel belongs to object
 - Otherwise (i.e. all pixels of are background), set to background (no change)



85

Dilation

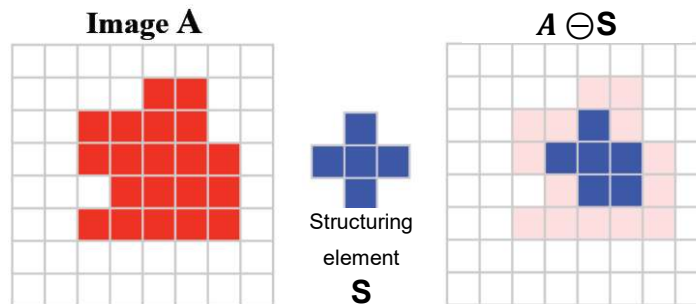
- As **max filter**
- Can be applied both on
 - binary images
 - or grayscale images



86

Erosion

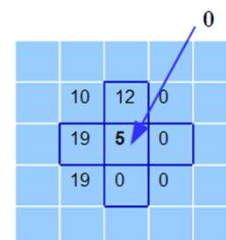
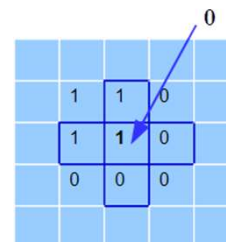
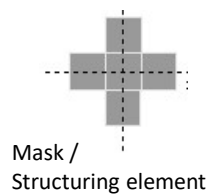
- We put the element S on each pixel x of A
 - like convolution
- If **all pixels of S are onto object pixels (A)**, then the **central pixel** belongs to **object**
 - Otherwise (i.e. a mask pixel is background), set to background



87

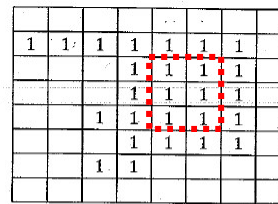
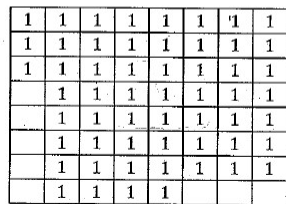
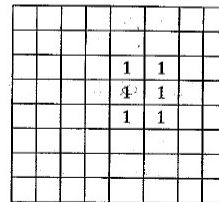
Erosion

- As **min filter**
- Can be applied both on
 - binary images
 - or grayscale images



88

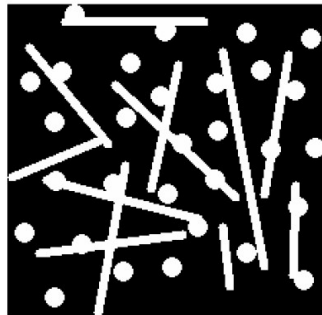
2D example

(a) Binary image B (b) Structuring element S (c) Dilation $B \oplus S$ (d) Erosion $B \ominus S$

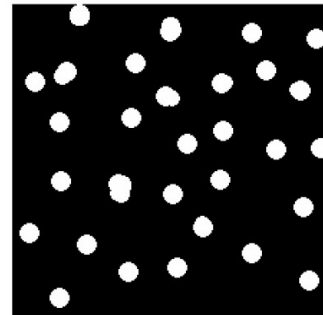
89

Opening

- Erode, then dilate
- Remove small objects, keep original shape



Before opening

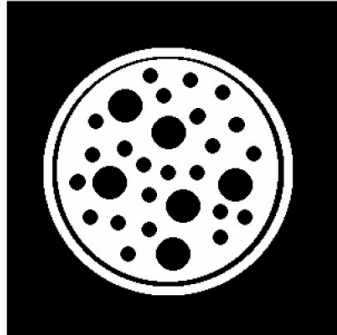


After opening

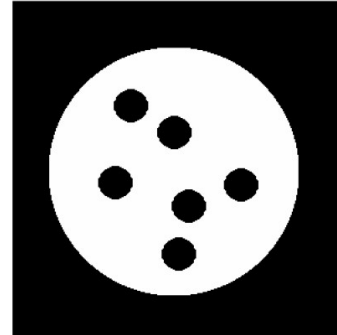
90

Closing

- Dilate, then erode
- Fill holes, but keep original shape



Before closing

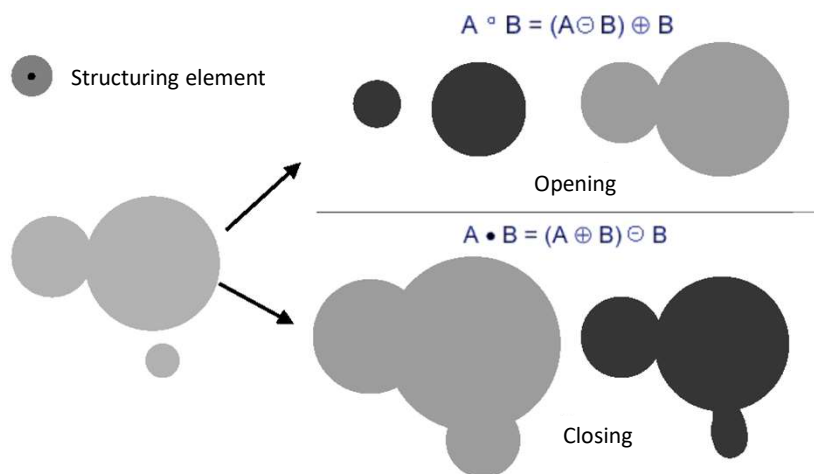


After closing

demo: <http://bigwww.epfl.ch/demo/jmorpho/start.php>

91

Opening vs Closing



92



HUST

hust.edu.vn fb.com/dhbkhn

Contents

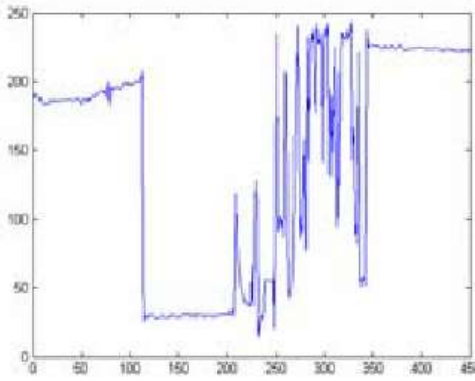
- Remind: digital image representation
- Point Operators
- Convolution and spatial filtering
- More neighborhood operators
- Image transforms
 - Frequency domain

CV Group – School of Information and Communication and Technology

93

Frequencies in images

Intensity profile



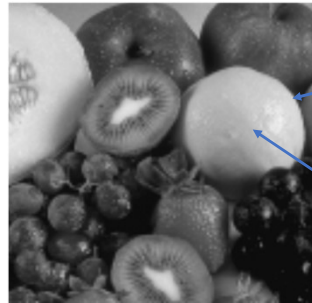



94

94

Frequencies in images

- What are the (low/high) frequencies in an image?
 - Frequency = **intensity change**
 - Slow changes (homogeneous /blur regions): **low frequency**
 - fast/abrupt changes (edge, contour, noise): **high frequency**

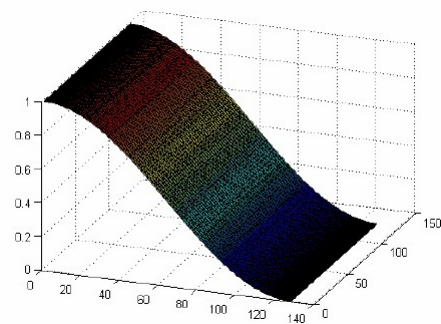
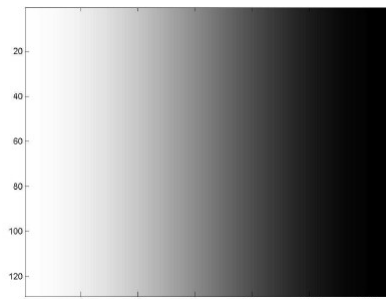


High frequency

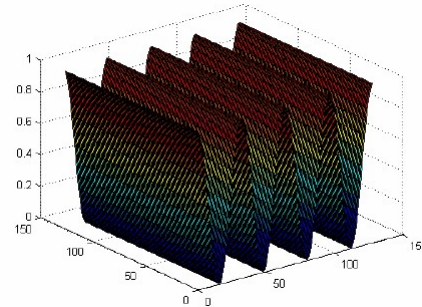
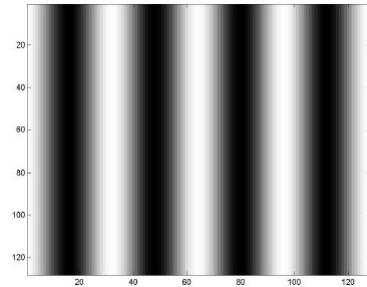
Low frequency

Most of energy concentrated in low frequencies

Low frequencies



High frequencies



97

Image spectral analysis

- An image is a visual signal
 - We can analyse the frequencies of the signal
- How?
 - We will create a new « image » which will contains all frequencies of the image
 - Like a 2D frequency graphic
 - The basic tool for it is the **Fourier Transform**
- We talk about the **frequency domain**, opposing to the **spatial domain** (image)

98

Frequencies in a signal

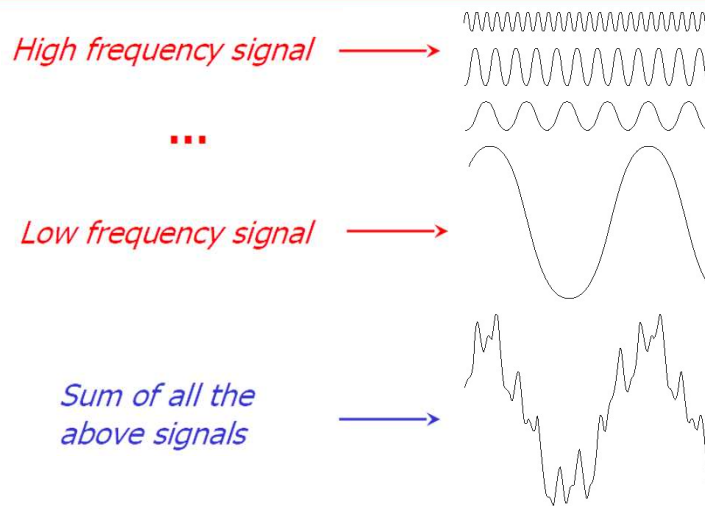


FIGURE 4.1 The function at the bottom is the sum of the four functions above it. Fourier's idea in 1807 that periodic functions could be represented as a weighted sum of sines and cosines was met with skepticism.

Fourier series

A bold idea (1807) - Jean Baptiste

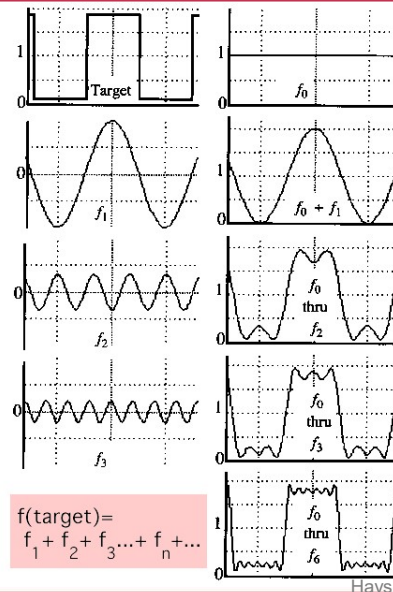
Joseph Fourier (1768-1830):

Any univariate function can be rewritten as a **weighted sum of sines and cosines** of different frequencies.

Our building block:

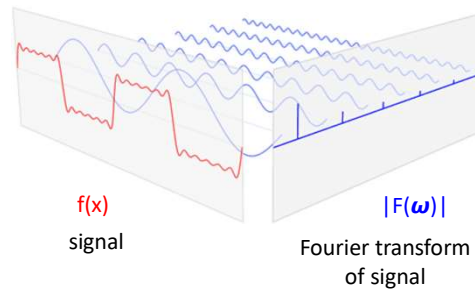
$$A \sin(\omega t) + B \cos(\omega t)$$

Add enough of them to get any signal $g(t)$ you want!



Fourier Transform

- Fourier transform is a mathematical transform that
 - Decomposes functions depending on space or time into functions depending on spatial or temporal frequency



The math

$$\text{Fourier Transform : } F(\omega) = \int_{-\infty}^{+\infty} f(x)e^{-i\omega x} dx$$

$$\text{Inverse Fourier Transform : } f(x) = \frac{1}{2\pi} \int_{-\infty}^{+\infty} F(\omega)e^{i\omega x} d\omega$$

- Where are the sines and cosines? $e^{i\omega x} = \cos(\omega x) + i \sin(\omega x)$
- The result is a complex function $F(\omega) = R(\omega) + iI(\omega)$
- We've been showing only the **amplitude A (spectrum)** so far:
- Phase is also encoded: $A = \pm \sqrt{R(\omega)^2 + I(\omega)^2}$

$$\phi = \tan^{-1} \frac{I(\omega)}{R(\omega)}$$

2D FFT

- Continuous FFT:

$$F(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{-2\pi i(xu + yv)} dx dy$$

- Inverse FFT:

$$f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) e^{2\pi i(xu + yv)} du dv$$

2D FFT - discrete

Direct transform

$$F(u, v) = \frac{1}{MN} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) \exp \left[-2\pi i \left(\frac{mu}{M} + \frac{nv}{N} \right) \right],$$

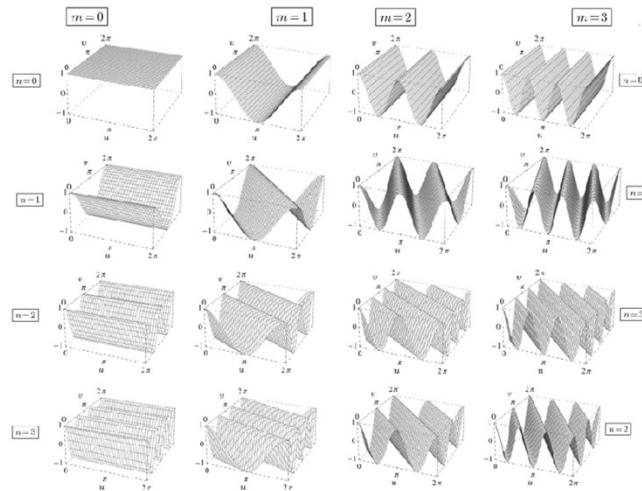
$$u = 0, 1, \dots, M-1, \quad v = 0, 1, \dots, N-1,$$

Inverse transform

$$f(m, n) = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) \exp \left[2\pi i \left(\frac{mu}{M} + \frac{nv}{N} \right) \right],$$

$$m = 0, 1, \dots, M-1, \quad n = 0, 1, \dots, N-1.$$

Function cosin 2D



105

Separability of 2D DFT

- Notice that Fourier transform “filter elements” can be expressed as products

$$\exp \left[2\pi i \left(\frac{xu}{M} + \frac{yv}{N} \right) \right] = \underbrace{\exp \left[2\pi i \frac{xu}{M} \right]}_{\text{2D DFT}} \underbrace{\exp \left[2\pi i \frac{yu}{M} \right]}_{\text{1D DFT (row)}} \underbrace{\exp \left[2\pi i \frac{yv}{N} \right]}_{\text{1D DFT (column)}}$$

- Formula above can be broken down into simpler formulas for 1D DFT

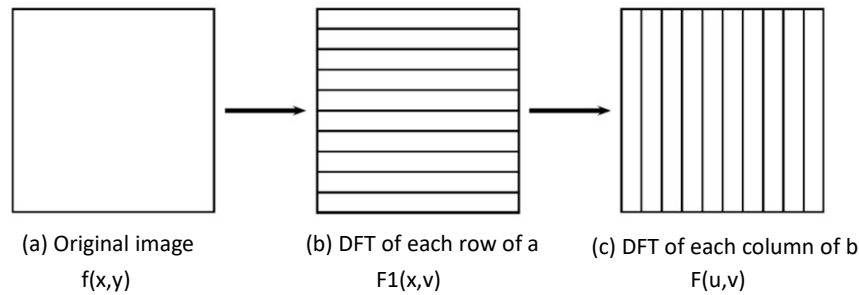
$$F(u) = \sum_{x=0}^{M-1} f(x) \exp \left[-2\pi i \frac{xu}{M} \right],$$

$$f(x) = \frac{1}{M} \sum_{u=0}^{M-1} F(u) \exp \left[2\pi i \frac{xu}{M} \right]$$

106

Separability of 2D DFT

- Using their separability property, can use 1D DFTs to calculate rows then columns of 2D Fourier Transform



Properties of 2D DFT

- Linearity: DFT of a sum is equal to sum (or multiplication) of the individual DFT's
- Cyclicity (frequencies)
 - Cycle?
 - $F(u,v) = F(u+a*M, v+b*N)$ a, b are integers

$$\begin{aligned}
 F(u + a * M, v + b * N) &= \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) * \exp \left[-2\pi i \left(\frac{xu}{M} + a + \frac{yv}{N} + b \right) \right] \\
 &= \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) * \exp \left[-2\pi i \left(\frac{xu}{M} + \frac{yv}{N} \right) \right] * \exp[-2\pi i(ax + by)]
 \end{aligned}$$

$$\exp[-2\pi i(ax + by)] = \cos[2\pi(ax + by)] + \sin[2\pi(ax + by)] = 1$$

DC Component

- Recall that

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \exp \left[-2\pi i \left(\frac{xu}{M} + \frac{yv}{N} \right) \right]$$

- The value $F(0,0)$ of DFT is called DC coefficient
- If we put $u=v=0$ then

$$F(0, 0) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \exp(0) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y)$$

- Essentially $F(0,0)$ is the sum of all terms in the original matrix

Image Fourier transform

Original image

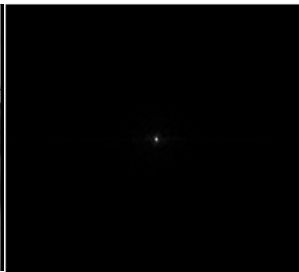
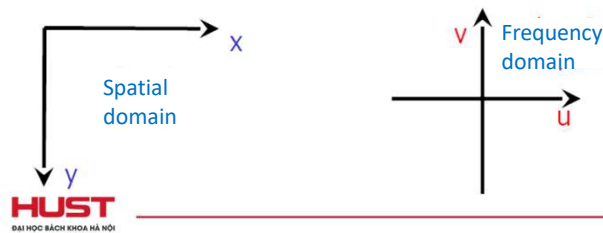
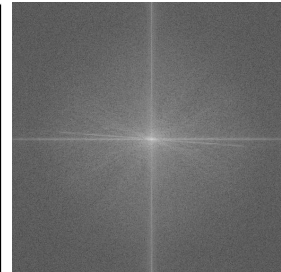
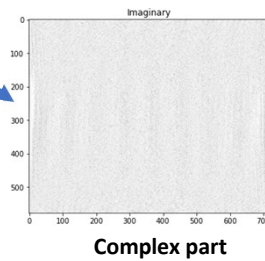
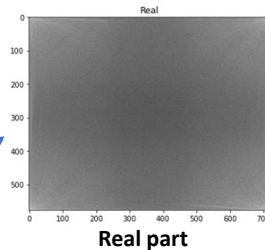
Spectra $|F(u, v)|$ Enhanced Spectra
 $\log(1 + |F(u, v)|)$ 

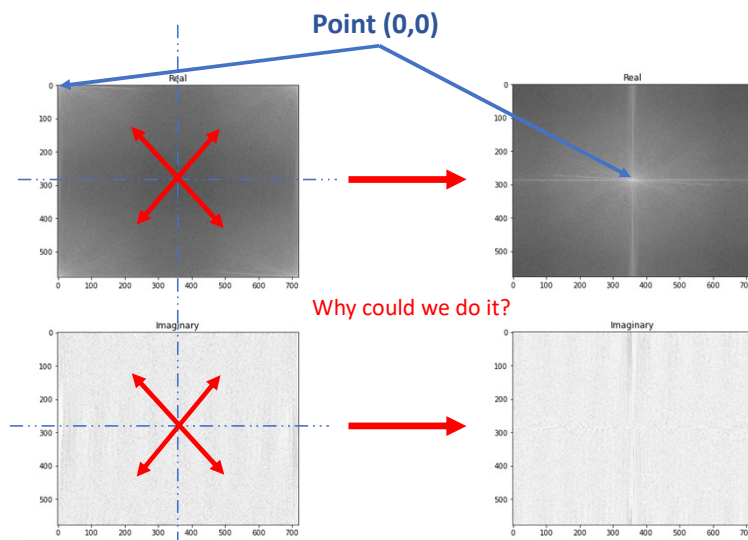
Image Fourier transform

$$F(u,v) = R(u,v) + i \cdot I(u,v)$$



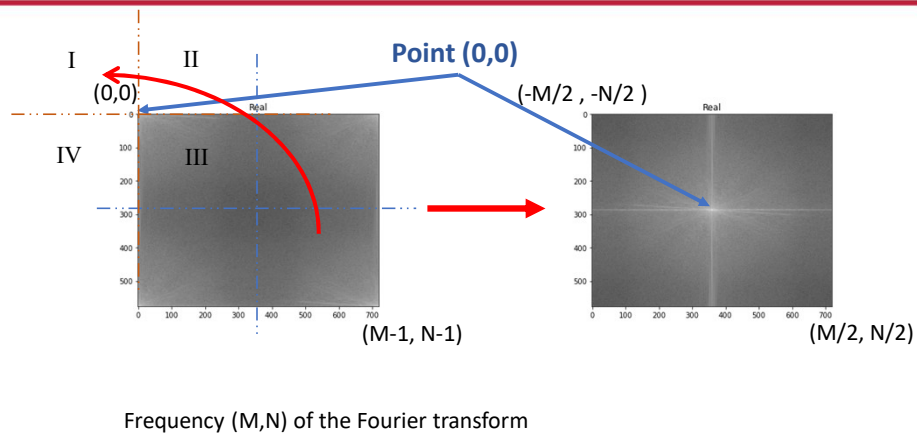
111

Image Fourier transform



112

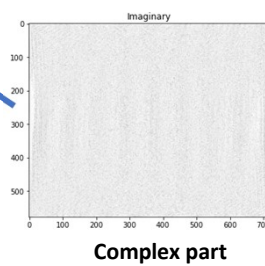
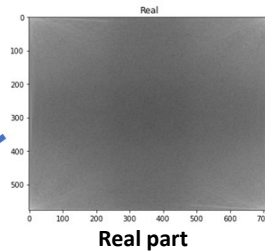
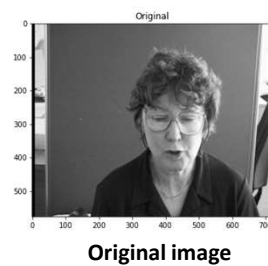
Image Fourier transform



113

Image reconstruction from frequency images

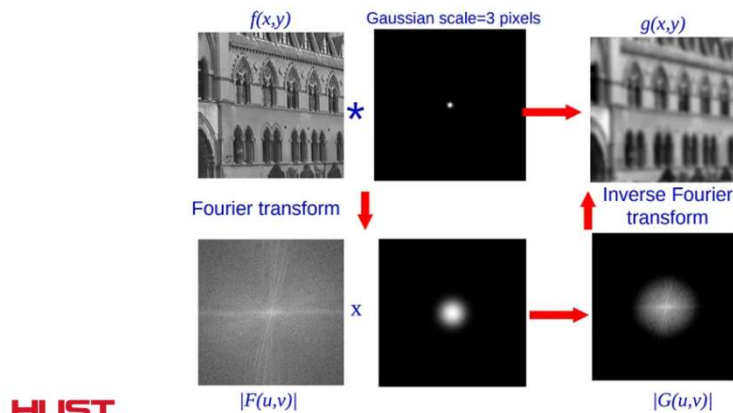
$$F(u,v) = R(u,v) + i \cdot I(u,v)$$



114

Convolution using 2D DFT

- DFT allows “convolution” by a different method
- “Convolution” becomes multiplication



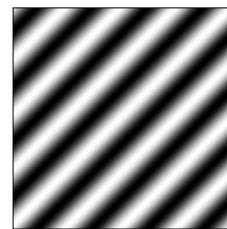
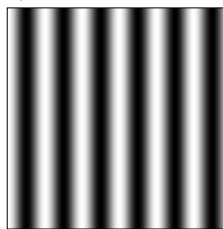
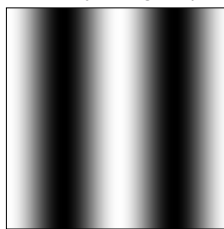
HUST
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

115

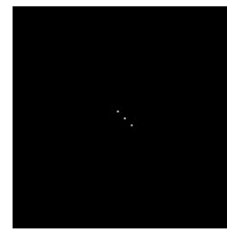
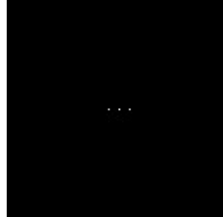
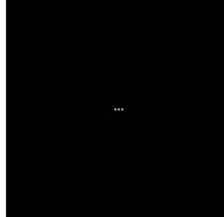
115

Fourier analysis in images

Intensity images (spatial domain)



Fourier images (spectral images – amplitude images)



<http://sharp.bu.edu/~slehar/fourier/fourier.html#filtering>
More: <http://www.cs.unm.edu/~brayer/vision/fourier.html>

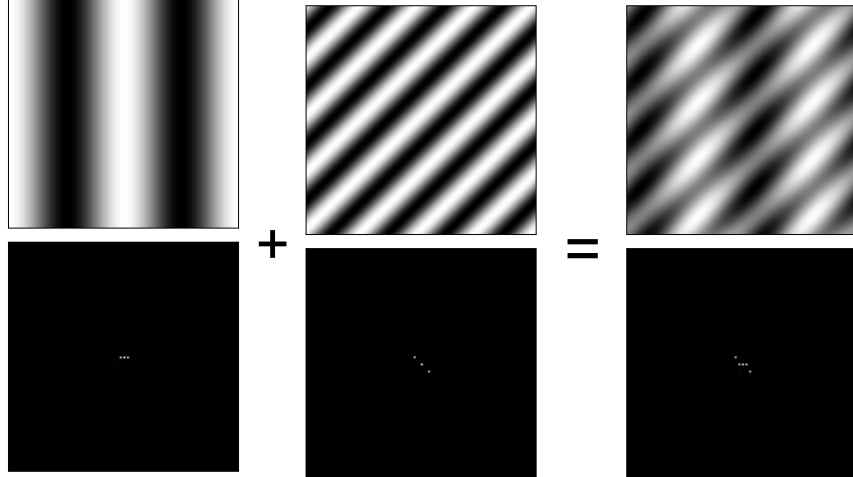
HUST
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

116

116

Signals can be composed

Intensity images (spatial domain)



Fourier images (spectral images – amplitude images)

117

Fourier Transform of an image

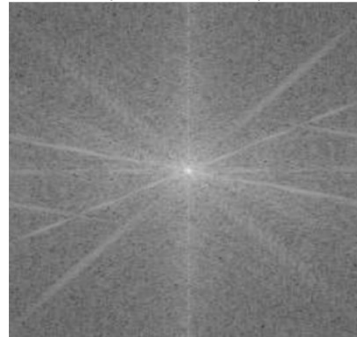
Natural image



$f(x,y)$

Fourier decomposition

Frequency coefficients (amplitude)



$|F(\omega)|$

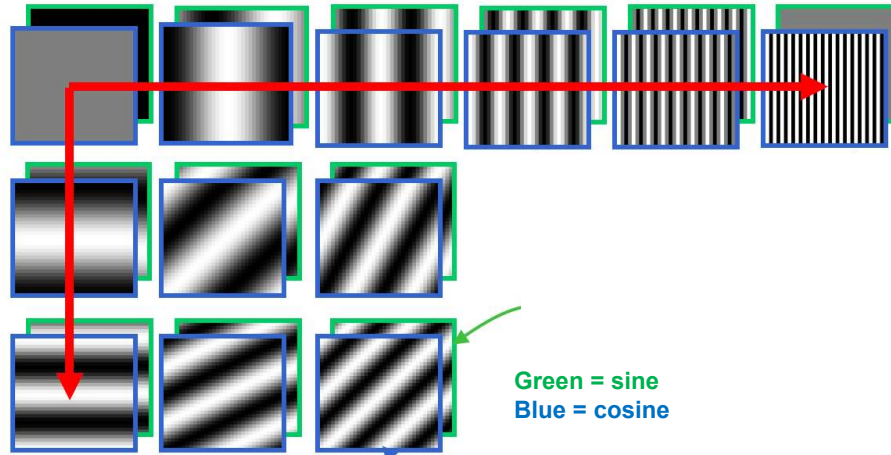
What does it mean to be at pixel x,y ?

What does it mean to be more or less bright in the Fourier decomposition image?

118

Fourier Bases

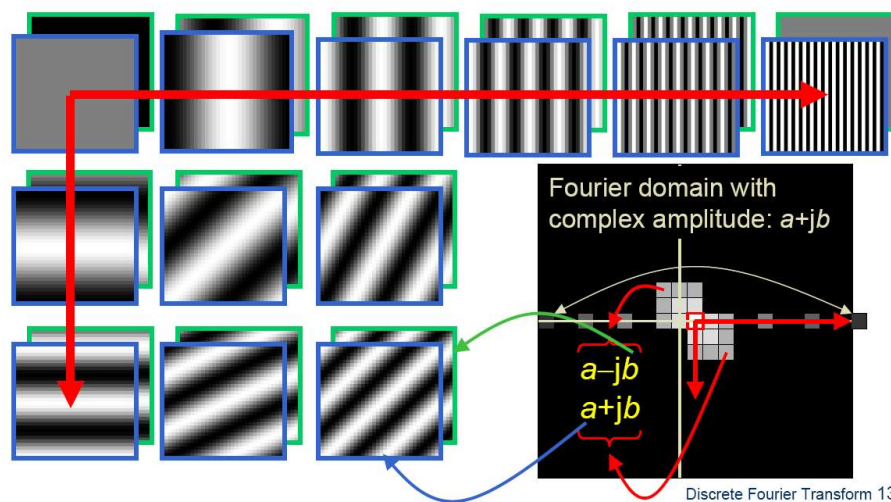
Teases away 'fast vs. slow' changes in the image.



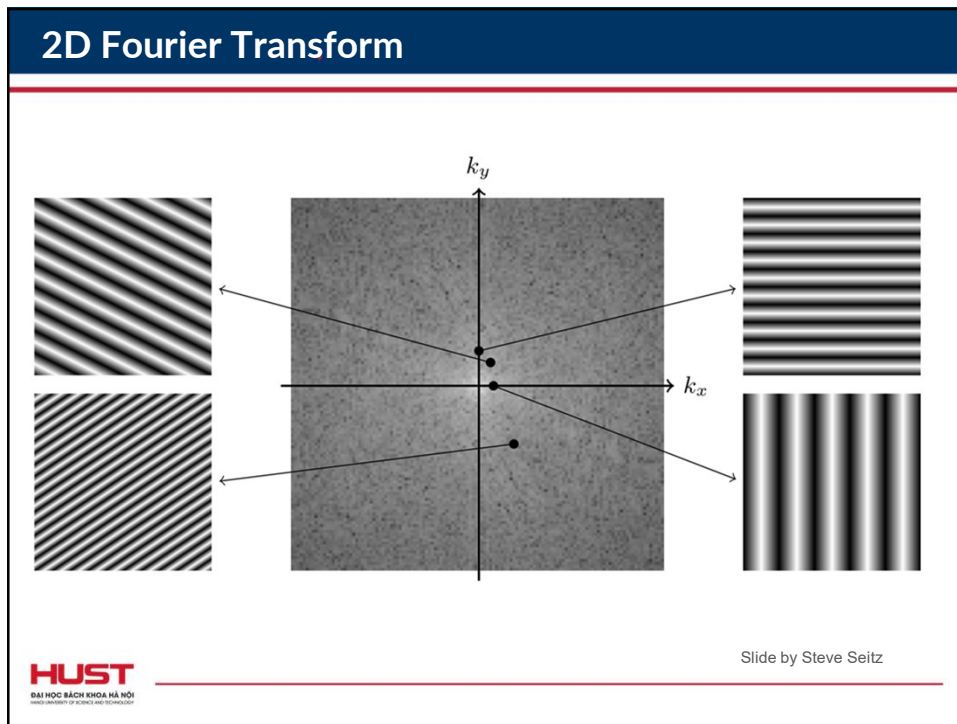
This change of basis is the Fourier Transform

119

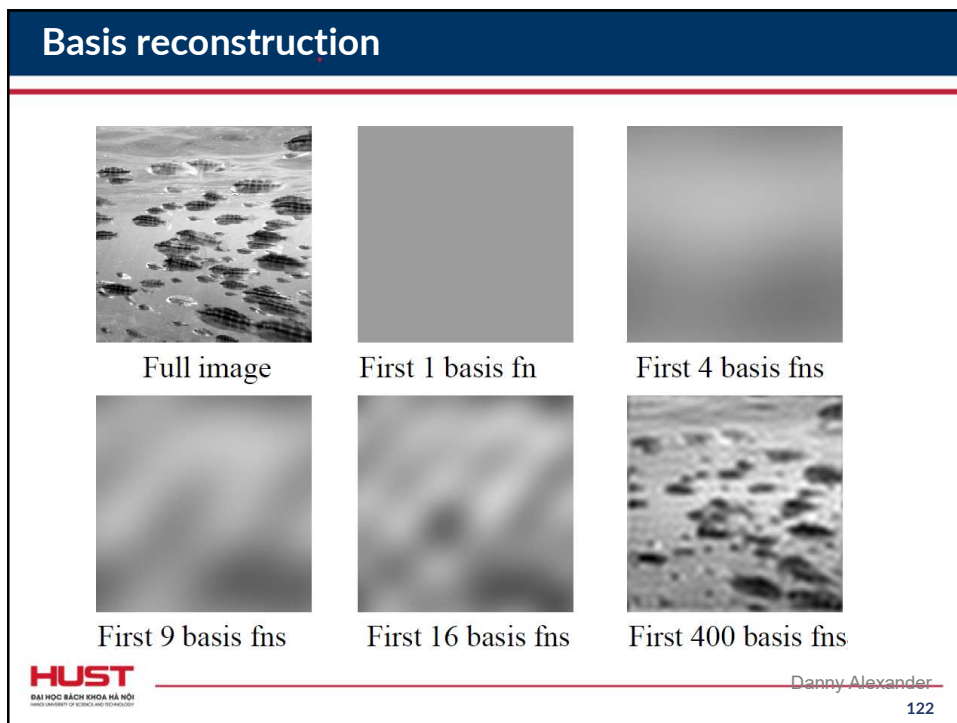
Fourier Bases



120

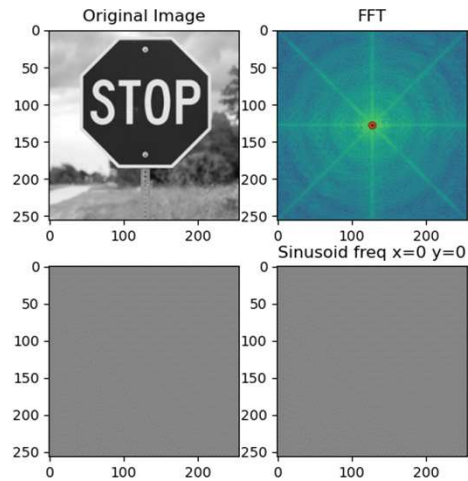


121



122

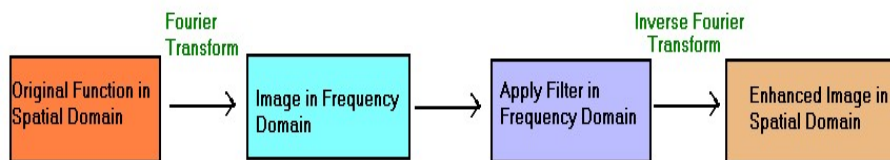
Basis reconstruction



123

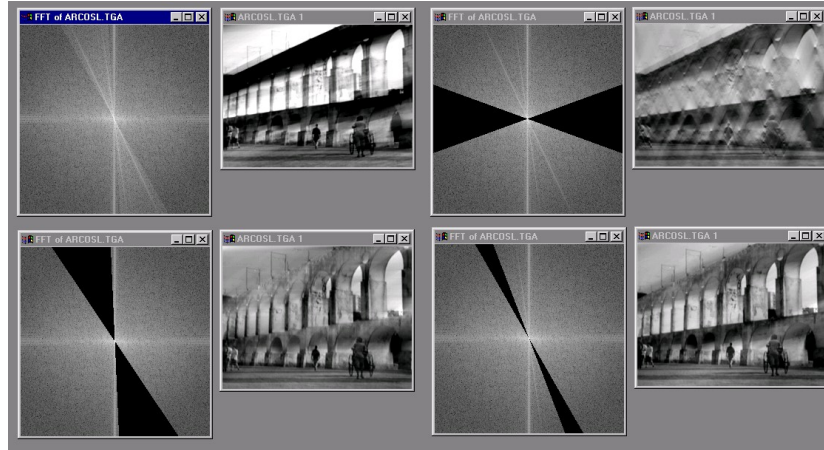
Image filtering in the frequential domain

- We will be dealing only with functions (images) of finite duration so we will be interested only in Fourier Transform



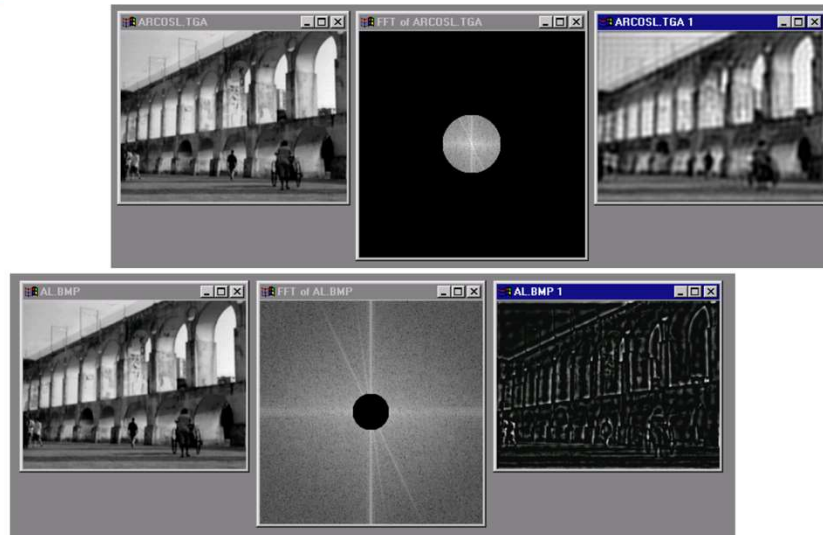
124

Now we can edit frequencies!



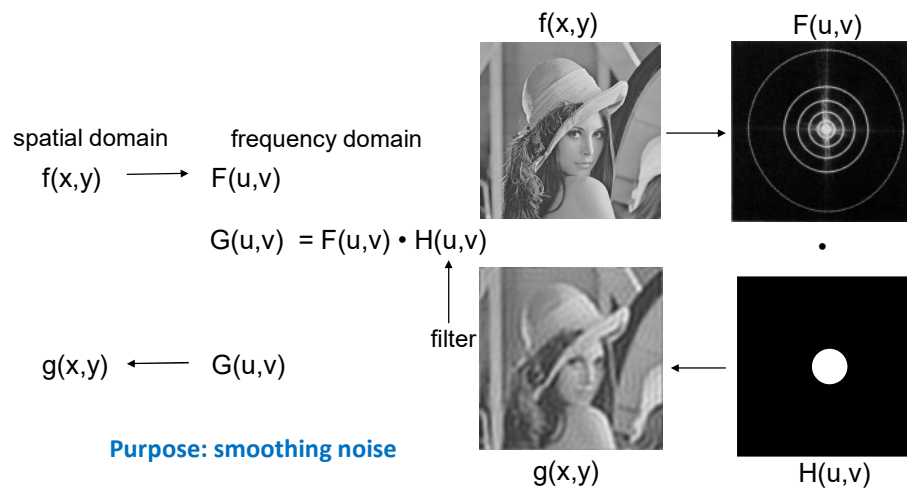
125

Low-pass and high-pass filtering



126

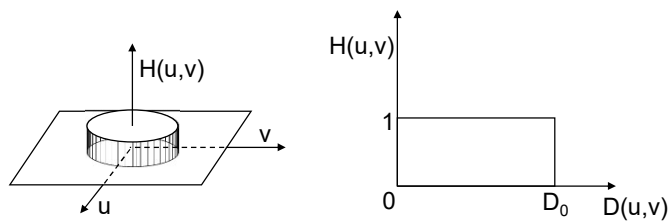
Low-pass filter



127

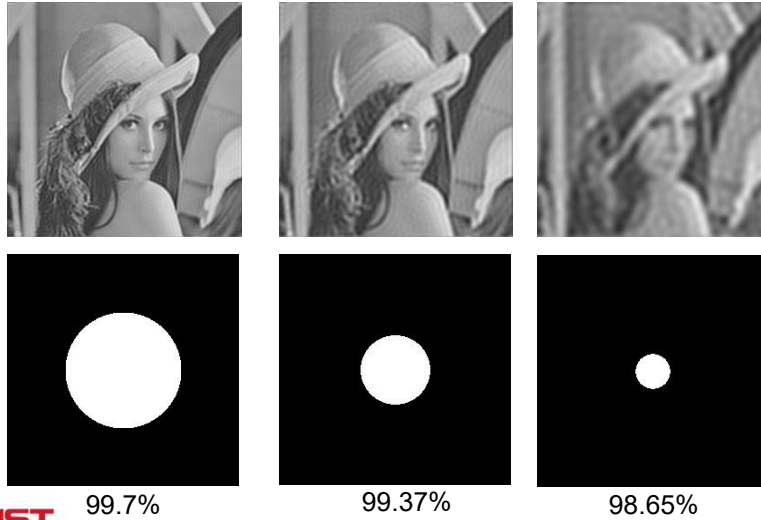
$H(u,v)$ - Ideal low-pass filter

$$H(u,v) = \begin{cases} 1 & D(u,v) \leq D_0 \\ 0 & D(u,v) > D_0 \end{cases} \quad \begin{aligned} D(u,v) &= \sqrt{u^2 + v^2} \\ D_0 &= \text{cut off frequency} \end{aligned}$$



128

Blurring - Ideal low-pass filters



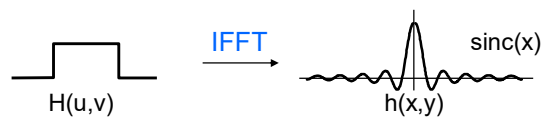
129

The ringing problem

$$G(u,v) = F(u,v) \cdot H(u,v)$$

↓ Convolution Theorem

$$g(x,y) = f(x,y) * h(x,y)$$

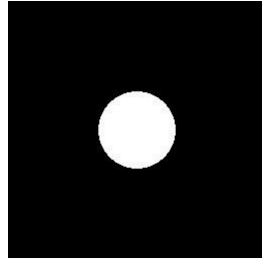


↑ D_0 ↓ Ringing radius + ↓ blur

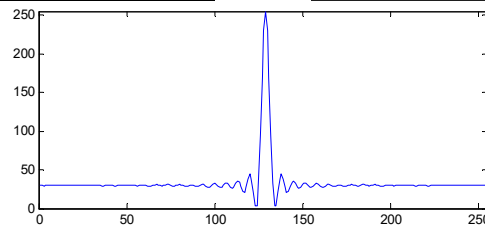
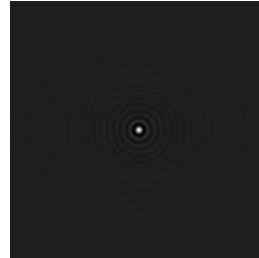
130

The ringing problem

Freq. domain

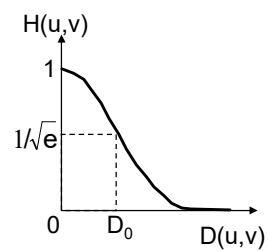
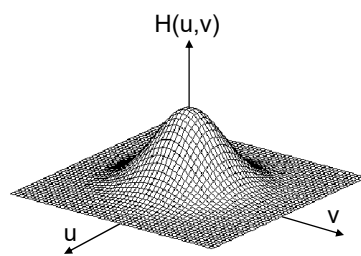


Spatial domain



131

$H(u,v)$ - Gaussian filter



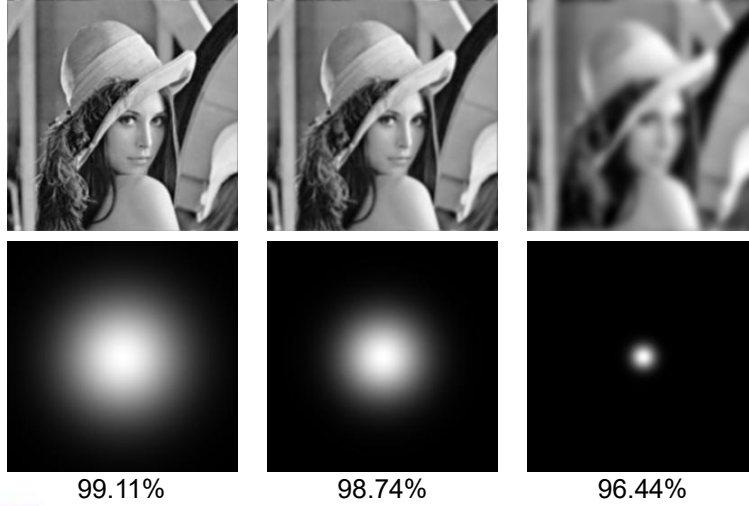
$$H(u,v) = e^{-D^2(u,v)/(2D_0^2)}$$

$$D(u,v) = \sqrt{u^2 + v^2}$$

Softer Blurring + no Ringing

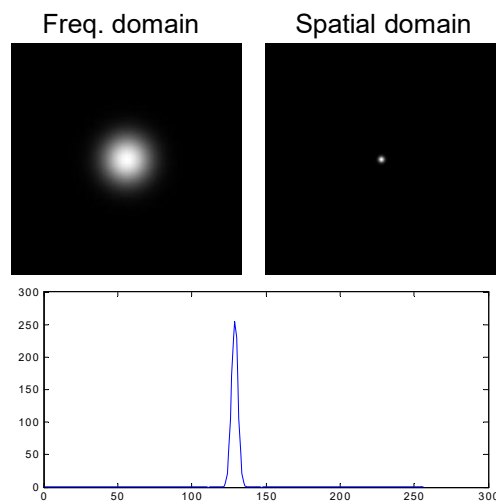
132

Blurring - Gaussain lowpass filter



133

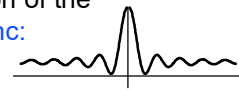
The Gaussian lowpass filter



134

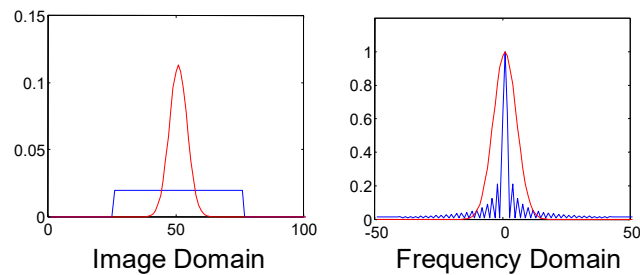
Blurring in the Spatial Domain

Averaging = convolution with $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$ = point multiplication of the transform with **sinc**:



Gaussian Averaging = convolution with $\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$

= point multiplication of the transform with **a gaussian**.

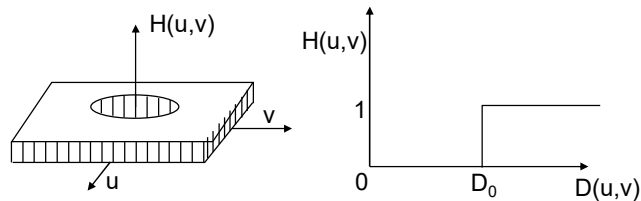


135

High-pass filter

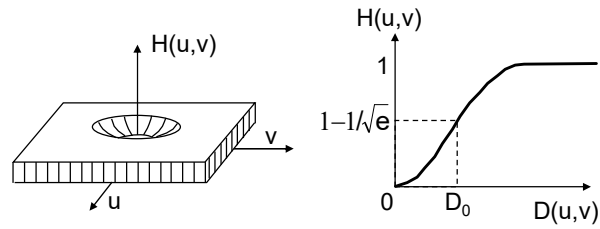
$H(u,v)$ - Ideal Filter

$$H(u,v) = \begin{cases} 0 & D(u,v) \leq D_0 \\ 1 & D(u,v) > D_0 \end{cases} \quad \begin{aligned} D(u,v) &= \sqrt{u^2 + v^2} \\ D_0 &= \text{cut off frequency} \end{aligned}$$



136

High-pass gaussian filter

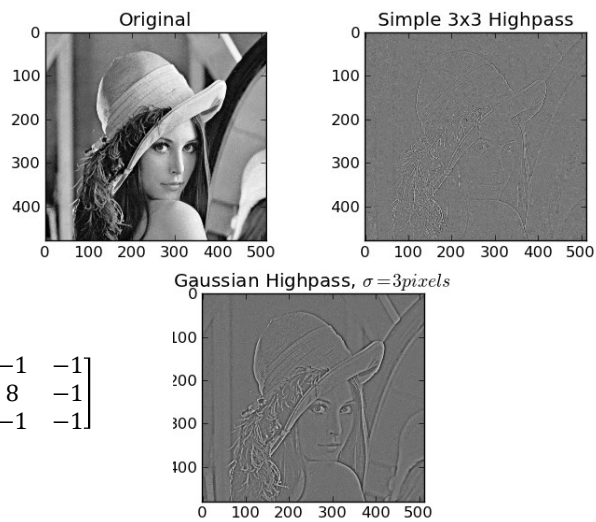


$$H(u,v) = 1 - e^{-D^2(u,v)/(2D_0^2)}$$

$$D(u,v) = \sqrt{u^2 + v^2}$$

137

High-pass filtering



$$h_1(3 \times 3) = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

138

High pass filtering

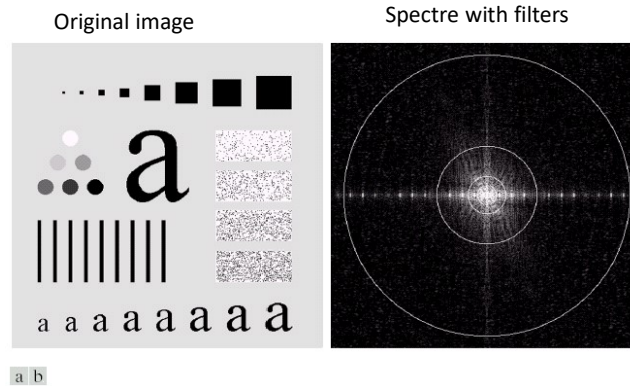


FIGURE 4.11 (a) An image of size 500×500 pixels and (b) its Fourier spectrum. The superimposed circles have radii values of 5, 15, 30, 80, and 230, which enclose 92.0, 94.6, 96.4, 98.0, and 99.5% of the image power, respectively.

139

High pass filtering

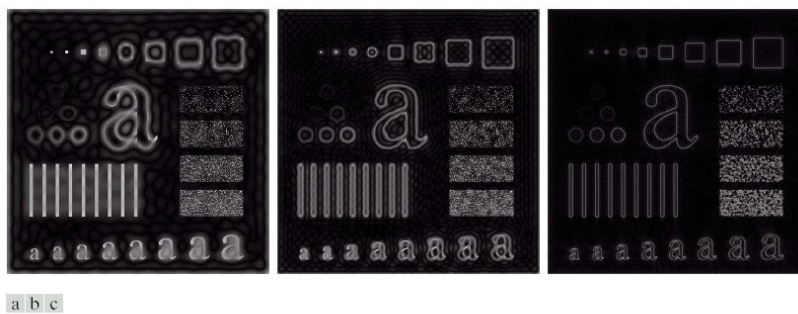
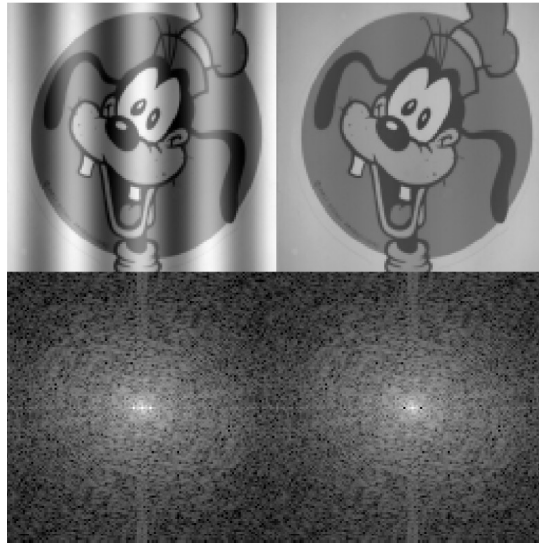


FIGURE 4.24 Results of ideal highpass filtering the image in Fig. 4.11(a) with $D_0 = 15, 30$, and 80 , respectively. Problems with ringing are quite evident in (a) and (b).

140

Removing sinus noise

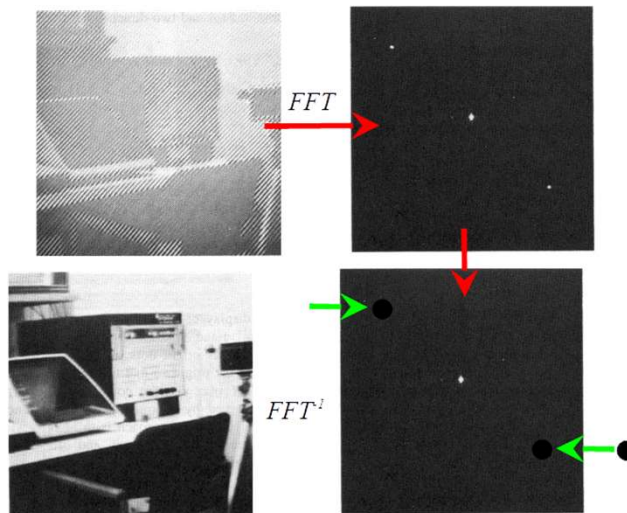


HUST
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Brayer

141

Removing sinus noise



HUST
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

142

Hybrid Images

A. Oliva, A. Torralba, P.G. Schyns, SIGGRAPH 2006

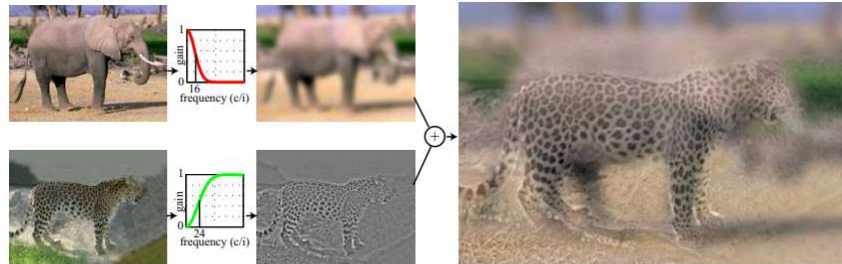


Figure 2: hybrid images are generated by superimposing two images at two different spatial scales: the low-spatial scale is obtained by filtering one image with a low-pass filter, and the high spatial scale is obtained by filtering a second image with a high-pass filter. The final hybrid image is composed by adding these two filtered images.



HUST

hust.edu.vn
fb.com/dhbkhn

THANK YOU !

CV Group – School of Information and Communication and Technology